IU International University of Applied Sciences

Degree programme: Applied Artificial Intelligence

PROJECT REPORT · Task 1

Handwritten Digit Recognition with Convolutional Neural Networks

Project: Computer Vision · DLBAIPCV01

SUBMITTED BY

Mohsen Nasiri

Matriculation number: 32206338

TUTOR

Gissel Velarde

Submission date: 13 October 2026

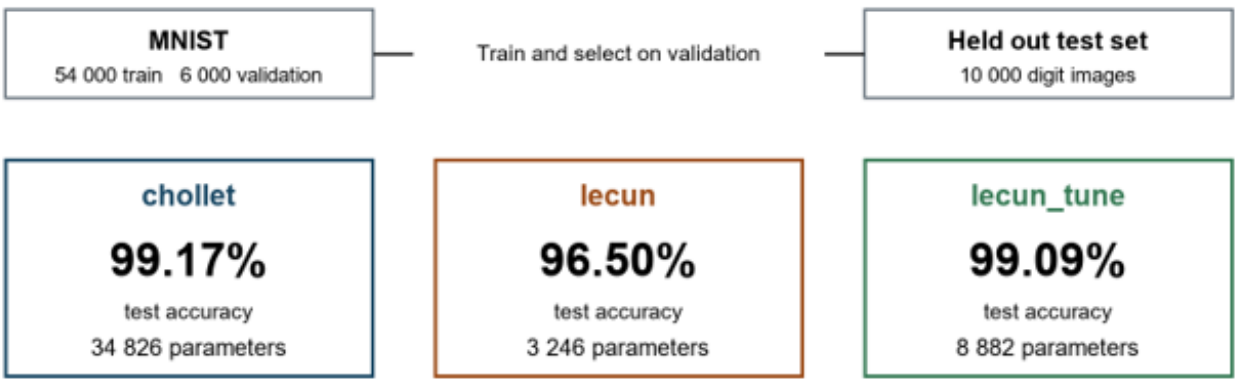
Abstract

This project investigates how closely a compact approximation of the convolutional neural network described by LeCun et al. (1989) can match the performance of a modern Keras-based handwritten digit classifier. Three models are evaluated using the MNIST dataset: the Chollet baseline, a historical approximation of LeCun's architecture, and a tuned adaptation of that approximation. Throughout this report, these models are referred to as Chollet, LeCun, and LeCun Tuned, respectively. The objective is to compare their architectural choices, predictive performance, model complexity, and training costs.

All three models use the same training, validation, and test sets, although their preprocessing procedures differ. The historical approximation retains four and twelve convolutional feature maps, 5 × 5 kernels, tanh activations, and alternating average-pooling stages. Modern adaptations include full channel connectivity, spatial dropout, Adam optimisation, and softmax classification. Ten manually specified hyperparameter configurations are evaluated, varying input resolution, regularisation, network capacity, and training settings. The final configuration is selected according to validation loss.

The models achieve test accuracies of 99.17% for Chollet, 96.50% for LeCun, and 99.09% for LeCun Tuned. The findings demonstrate that tuning substantially improves the historical approximation while retaining a smaller parameter count. However, Chollet achieves the highest observed accuracy, and the results do not establish generalisation to other handwriting datasets or consistency across multiple random seeds.

Fig. 1: Test accuracy and trainable parameters of the three implemented models



Source: Own illustration based on the recorded experimental results.

Contents

Abstract II

List of figures IV

List of tables IV

Abbreviations IV

1 Introduction and project planning 1

1.1 Motivation and objective 1

1.2 Scope and sequence 1

2 Dataset and experimental method 2

3 Historical approximation and design choices 3

4 Hyperparameter search and model selection 4

4.1 Search configuration and ranking 4

4.2 Observations from the trials 5

5 Comparative results and learning behaviour 5

6 Error analysis and application example 7

6.1 Incorrect predictions 7

6.2 Comparison of the remaining errors 8

6.3 Recognition of handwritten photographs 8

7 Reflection and conclusion 9

References 11

Appendix A Reproduction and evidence 12

Evidence files 12

Reproduction procedure 12

List of figures

Fig. 1: Test accuracy and trainable parameters of the three implemented models II
Fig. 2: Examples of the ten MNIST digit classes 2
Fig. 3: Architectures of the LeCun approximation and LeCun Tuned models 2
Fig. 4: Validation loss over training epochs for the three models 6
Fig. 5: Examples of incorrect test predictions from Chollet, LeCun, and LeCun Tuned 7
Fig. 6: Recall by digit class for the three models 8
Fig. 7: Detected handwritten digit regions in the two input photographs 9

List of tables

Tab. 1: Comparison of the historical LeCun architecture and the implemented baseline models 3
Tab. 2: Hyperparameter configurations ranked by validation loss 4
Tab. 3: Test performance, parameter counts, and training costs of the three models 6
Tab. 4: Predicted digit sequences and mean confidence scores for the handwritten photographs 9

Abbreviations

CNN Convolutional neural network

CPU Central processing unit

MNIST Modified National Institute of Standards and Technology dataset

ReLU Rectified linear unit

1 Introduction and project planning

1.1 Motivation and objective

Handwritten digit recognition involves identifying numerical symbols from images despite variations in stroke thickness and shape. This project focuses on classifying individual handwritten digits and demonstrates an additional application in which multiple digits are extracted and recognised from photographs.

The historical network described by LeCun et al. (1989) uses local receptive fields, shared convolutional weights, and successive spatial reduction to recognise handwritten digits. Its architecture consists of four hidden stages alternating between convolution and subsampling (LeCun et al., 1989, pp. 400–402). This architecture serves as the foundation for the historical approximation implemented in this project. For comparison, Chollet's modern Keras CNN example is used as a baseline, employing two convolutional layers, max pooling, dropout, and a ten-class output (Chollet, 2020, “Build the model” section).

The primary objective is to investigate whether a compact approximation of LeCun's historical architecture can achieve competitive recognition performance on MNIST. A further objective is to determine how much its performance can be improved through hyperparameter tuning while maintaining a lower parameter count than the Chollet baseline. The three models are therefore compared in terms of classification accuracy, loss, model complexity, and training time.

1.2 Scope and sequence

The project follows four main stages. First, the experimental environment is established, including the required libraries, configuration settings, and functions. Second, the Chollet baseline and LeCun historical approximation are implemented, trained, and evaluated. Third, the historical approximation is extended through hyperparameter search, combining four initial trials with six additional configurations. Finally, the three models are compared using their recorded performance metrics, learning behaviour, and classification errors.

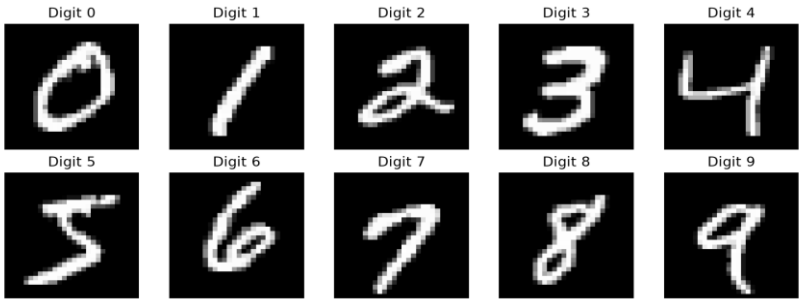
The experiments are conducted on a local CPU, where training time and computational resources are practical constraints. To manage these limitations, tuning trials are restricted to a maximum of 30 epochs, and completed models, training histories, and evaluation results are saved for reuse.

The implementation is organised in a Jupyter notebook, separating libraries, variables, reusable functions, model training, tuning, and result analysis. An additional computer vision application uses OpenCV to extract handwritten digits from photographs and applies the three trained models to recognise them. This demonstrates the use of the classifiers beyond the standard MNIST test images.

2 Dataset and experimental method

MNIST contains 60,000 development images and 10,000 test images of handwritten digits ranging from zero to nine. Each image consists of a 28 × 28 grayscale array with pixel intensity values between 0 and 255. For this experiment, the first 54,000 development images are used for training, while the remaining 6,000 form the validation set. This fixed, unstratified split is applied consistently across all three models, ensuring that each uses identical training, validation, and test samples. The official MNIST test set remains separate from the training and model selection procedures.

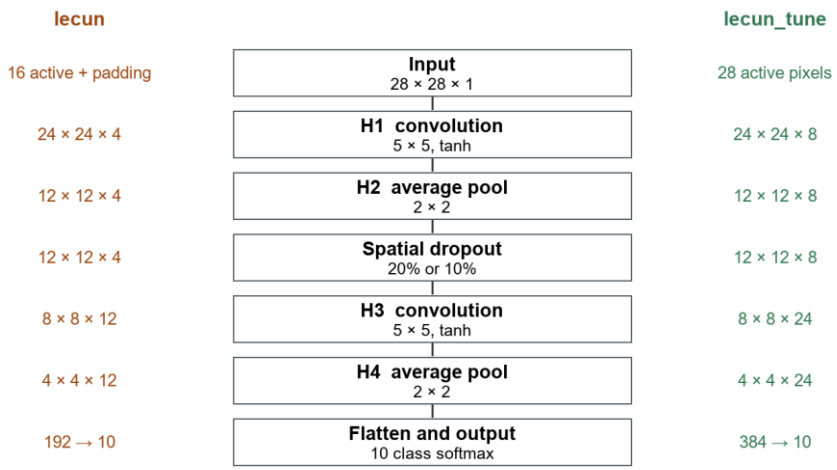
Fig. 2: Examples of the ten MNIST digit classes



Source: Own illustration using samples from the MNIST dataset (LeCun et al., 1998).

For the Chollet model, pixel values are normalised to the range 0 to 1 by dividing by 255. For the LeCun approximation, images are bilinearly resized to 16 × 16 pixels, padded with six pixels on each side, and mapped to the range −1 to 1. The LeCun Tuned model retains the original 28 × 28 resolution while using the same signed intensity range. Class labels are converted into ten-element one-hot encoded vectors. Since the models differ in both architecture and preprocessing, the comparison evaluates their complete configurations rather than isolating the effects of architecture alone.

Fig. 3: Architectures of the LeCun approximation and LeCun Tuned models



Source: Own illustration based on the implemented model architectures.

3 Historical approximation and design choices

Table 1 compares the original architecture described by LeCun et al. (1989) with the Chollet baseline and the implemented LeCun approximation. The comparison highlights differences in network structure, preprocessing, feature map connectivity, training methods, and parameter counts. The historical architecture serves as a reference for the design choices made in this project rather than a directly comparable experimental baseline.

Tab. 1: Comparison of the historical LeCun architecture and the implemented baseline models

Property	Chollet	Original LeCun	LeCun (implemented)
Data source	MNIST	Handwritten and printed digits	MNIST
Train / validation / test	54,000 / 6,000 / 10,000	9,840 / not reported / 2,707	54,000 / 6,000 / 10,000
Input canvas	28 × 28	16 × 16 active in 28 × 28	16 × 16 resized in 28 × 28
Hidden spatial stages	Conv, pool, conv, pool	H1, H2, H3, H4	Conv, pool, conv, pool
Convolution filters	32 then 64	4 then 12	4 then 12
Convolution kernel	3 × 3	5 × 5	5 × 5
Nonlinearity / pooling	ReLU / max 2 × 2	Sigmoidal / subsampling	tanh / average 2 × 2
H2 to H3 connections	All input channels	Selected map connections	All channels, spatial dropout 0.2
Output / loss	10 softmax / cross entropy	10 outputs / squared error	10 softmax / cross entropy
Training rule	Adam, initial rate 0.001	Second order back propagation	Adam, initial rate 0.001
Epochs / batch size	15 / 128	30 passes / not matched	15 / 128
Trainable parameters	34,826	2,578	3,246

Source: Own representation based on Chollet (2020), LeCun et al. (1989, pp. 397–402), and the implemented experimental configurations.

The implemented LeCun approximation preserves the historical architecture's four convolutional feature maps in the first stage, twelve in the second, 5 × 5 kernels, and alternating convolution and subsampling stages. However, several modifications were necessary to implement the architecture using standard Keras layers. Fixed average pooling replaces the original trainable subsampling operations, while standard convolutional layers connect all input feature maps to the following convolutional layer. Spatial dropout is introduced for regularisation, but it does not reproduce the original architecture's selective feature map connections.

The approximation also uses tanh activations, a ten-class softmax output, and categorical cross-entropy loss. Adam optimisation replaces the historical training procedure, providing a consistent optimiser for both initially implemented models. Additionally, MNIST images are resized to 16 × 16 pixels and padded to a 28 × 28 canvas, approximating the historical input representation without reproducing its original normalisation procedure. Since the historical network was evaluated using different data and training conditions, its reported recognition performance is not directly compared with the experimental results in this project (LeCun et al., 1989, pp. 397–398, 402).

The third model, LeCun Tuned, builds upon the implemented approximation while retaining its two convolutional and two pooling stages. The selected configuration increases the feature maps to 8 and 24, preserves the original 28 × 28 image resolution, and reduces spatial dropout from 0.2 to 0.1. These modifications aim to improve recognition performance while maintaining a compact architecture, rather than reproducing the historical network more faithfully. The tuning process and model selection are discussed in Section 4.

4 Hyperparameter search and model selection

4.1 Search configuration and ranking

The hyperparameter search evaluates ten manually specified configurations rather than an exhaustive grid search or Bayesian optimisation. Four previously completed trials are combined with six additional configurations to investigate the effects of input resolution, regularisation, network capacity, activation functions, pooling methods, and training settings. Each trial uses seed 42, with its configuration, trained weights, and learning history saved separately.

Tab. 2: Hyperparameter configurations ranked by validation loss

Configuration	Hyperparameters	Best epoch / run	Val loss	Val accuracy
wide shift	**8/24 maps, tanh, shifts**	**30 / 30**	**0.03054**	**99.22%**
wide relu max	8/24 maps, ReLU, max, shifts	16 / 22	0.03207	99.13%
wide relu	8/24 maps, ReLU	22 / 28	0.03515	99.00%
wide tanh	8/24 maps, tanh	26 / 30	0.03535	99.02%
small long	4/12 maps, rate .003	30 / 30	0.04043	98.80%
small batch64	4/12 maps, batch 64	30 / 30	0.04120	98.97%
v1 03	28 px, rate .003	14 / 15	0.04668	98.75%
v1 02	28 px, dropout .00	15 / 15	0.05077	98.60%
v1 01	16 px, dropout .00	15 / 15	0.08285	97.72%
v1 00	16 px, dropout .20	15 / 15	0.09877	96.87%

Source: Own representation based on the recorded hyperparameter tuning results.

The six additional trials use early stopping with a patience of six epochs and restore the weights associated with the lowest validation loss. A learning-rate reduction callback halves the learning rate after two consecutive epochs without improvement, subject to a minimum of 0.00005. Each trial is limited to 30 epochs. Table 2 distinguishes between the epoch achieving the lowest validation loss and the total number of epochs completed.

Random translation was introduced to help the model recognise digits despite small changes in their position, with the aim of improving generalisation. A translation factor of 0.07 was used, corresponding to approximately two pixels in a 28 × 28 image. Newly exposed pixels were filled with −1 to match the background after normalisation.

The selected configuration uses 8 and 24 convolutional filters, tanh activations, random translations, average pooling, spatial dropout of 0.1, a batch size of 128, and an initial learning rate of 0.001. This configuration achieves the lowest validation loss of 0.03054 and a validation accuracy of 99.22%.

4.2 Observations from the trials

The initial four trials demonstrate the effects of changing the input resolution and regularisation settings. With a 16 × 16 active image resolution, reducing spatial dropout from 0.2 to 0 decreases validation loss from 0.09877 to 0.08285 and increases validation accuracy from 96.87% to 97.72%. Increasing the active image resolution to 28 × 28 while maintaining zero dropout further reduces validation loss to 0.05077 and improves validation accuracy to 98.60%. These results suggest that preserving more image detail benefits the tested approximation.

A subsequent configuration combining a learning rate of 0.003 with spatial dropout of 0.1 achieves a lower validation loss of 0.04668, compared with 0.05077 for the preceding configuration. Since both the learning rate and dropout setting change, their individual contributions cannot be determined from this comparison alone.

The expanded search investigates larger networks containing 8 and 24 convolutional feature maps instead of 4 and 12. All four wider configurations achieve lower validation losses than the smaller configurations included in the search. Among the wider models, the unshifted ReLU configuration reaches its lowest validation loss of 0.03515 at epoch 22, with training stopping after epoch 28. The corresponding tanh configuration achieves a validation loss of 0.03535.

Introducing random translations to the wider tanh configuration further reduces validation loss to 0.03054, producing the best result among the tested configurations. The shifted ReLU configuration also performs better than its unshifted counterpart, although additional architectural settings differ between these trials.

Overall, the results favour retaining the original image resolution, increasing the number of convolutional feature maps, and introducing small positional variations during training. These observations explain the configuration selected for LeCun Tuned while preserving the principal convolution and pooling structure of the historical approximation.

5 Comparative results and learning behaviour

Table 3 summarises the test performance, parameter counts, and recorded training times of the three implemented models. All models are evaluated using the same 10,000 MNIST test images, allowing their predictive performance and computational requirements to be compared under a consistent experimental setup.

Tab. 3: Test performance, parameter counts, and training costs of the three models

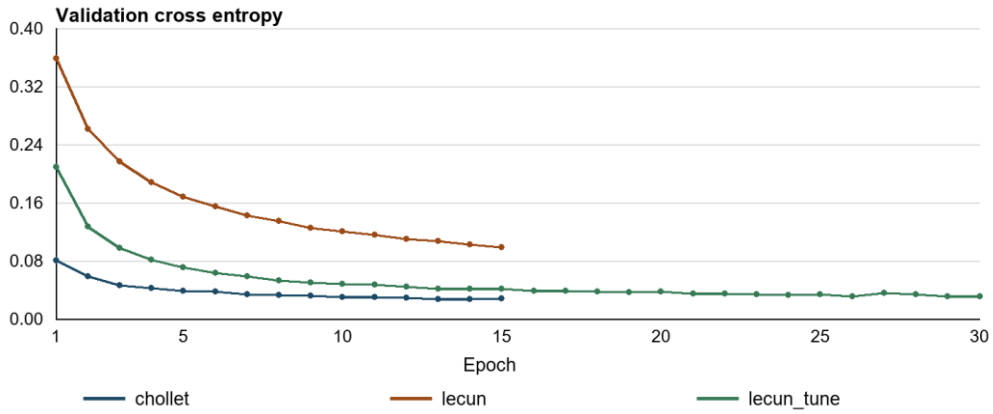
Model	Test loss	Accuracy	Parameters	Fit time (s)	Search fits (s)
Chollet	0.02596	99.17%	34,826	143.47	—
LeCun	0.10544	96.50%	3,246	48.62	—
LeCun Tuned	0.02842	99.09%	8,882	170.80	987.05

Source: Own representation based on recorded experimental results.

Chollet achieves the highest test accuracy of 99.17%, followed closely by LeCun Tuned at 99.09%, while the original LeCun approximation reaches 96.50%. These results correspond to 83, 91, and 350 incorrect predictions, respectively. Compared with LeCun, the tuned model improves accuracy by 2.59 percentage points and reduces the number of classification errors by 259. It also requires approximately 74.5% fewer trainable parameters than Chollet, with an accuracy difference of only 0.08 percentage points. However, the eight additional errors observed for LeCun Tuned do not establish a statistically significant difference without further testing.

Figure 4 compares the validation losses recorded during training, illustrating the learning behaviour of the three models.

Fig. 4: Validation loss over training epochs for the three models



Source: Own illustration based on recorded training histories.

The learning curves in figure 4 show that the initial LeCun approximation maintains a higher validation loss than the other two models. Chollet records a validation loss of 0.02779, while LeCun records 0.09877. The selected LeCun Tuned configuration achieves a minimum validation loss of 0.03054, demonstrating a substantial improvement over the initial approximation. The tuned model also trains over a longer period, allowing additional optimisation of its parameters.

Training time introduces another consideration. Chollet requires 143.47 seconds for 15 epochs, compared with 48.62 seconds for LeCun under the same epoch limit. Chollet therefore takes approximately 2.95 times as long to train, although it achieves higher recognition accuracy. The recorded times include training and validation but exclude preprocessing and final test evaluation.

LeCun Tuned requires 170.80 seconds for its selected 30-epoch training run, approximately 27.33 seconds longer than Chollet. Despite having considerably fewer parameters, its longer training

duration and data augmentation increase the total fitting time. This demonstrates that a smaller parameter count does not necessarily result in shorter training.

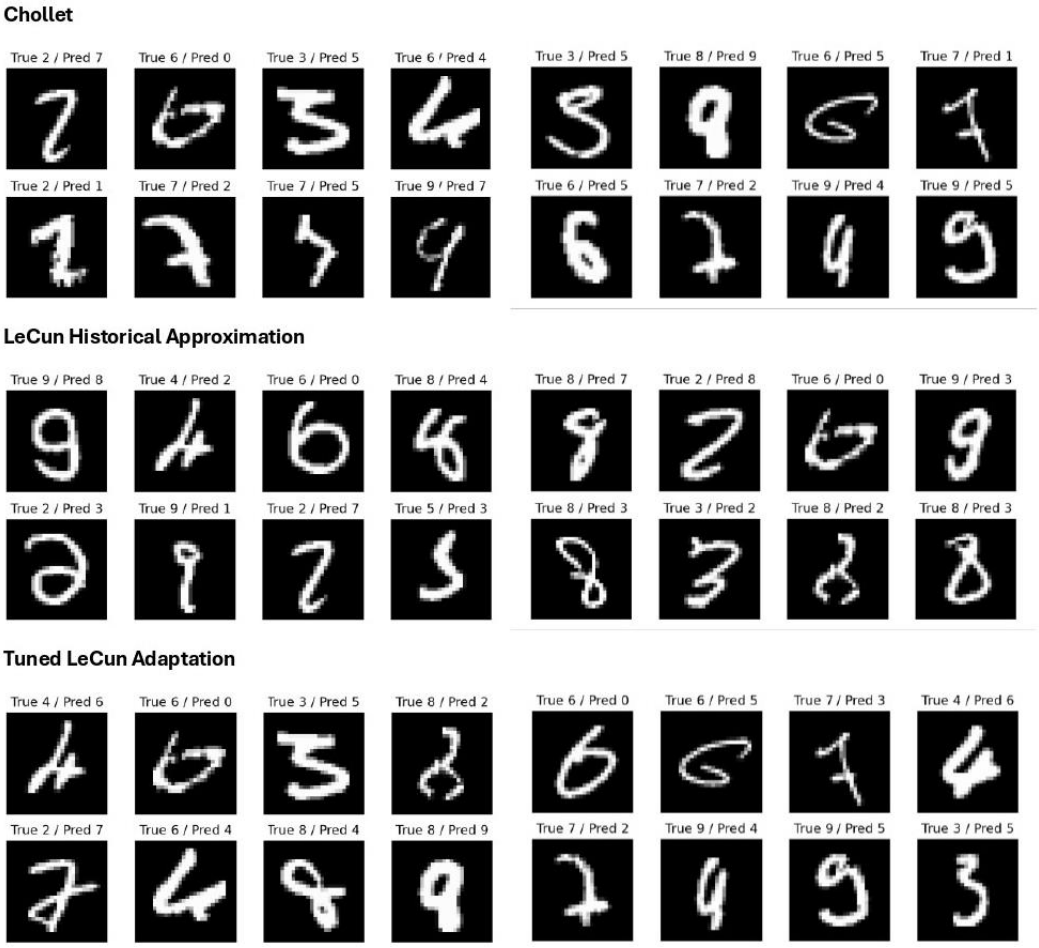
The complete hyperparameter search requires 988 seconds (approximately 16.45 minutes) across ten recorded trials. This includes 244 seconds for the four initial trials and 744 seconds for the six additional configurations. The selected model's training time is included in this search total. Consequently, the tuned architecture offers a reduction in model size without a corresponding reduction in the computational effort required for model development.

6 Error analysis and application example

6.1 Incorrect predictions

To examine the classification errors, the first 16 incorrectly predicted test images from each model are presented in Figure 5, together with their true and predicted labels. Several examples contain ambiguous handwriting, incomplete strokes, or digit shapes resembling other classes. These cases illustrate some of the difficulties encountered by the classifiers, although the selected images represent only a small subset of the complete test results.

Fig. 5: Examples of incorrect test predictions from Chollet, LeCun, and LeCun Tuned



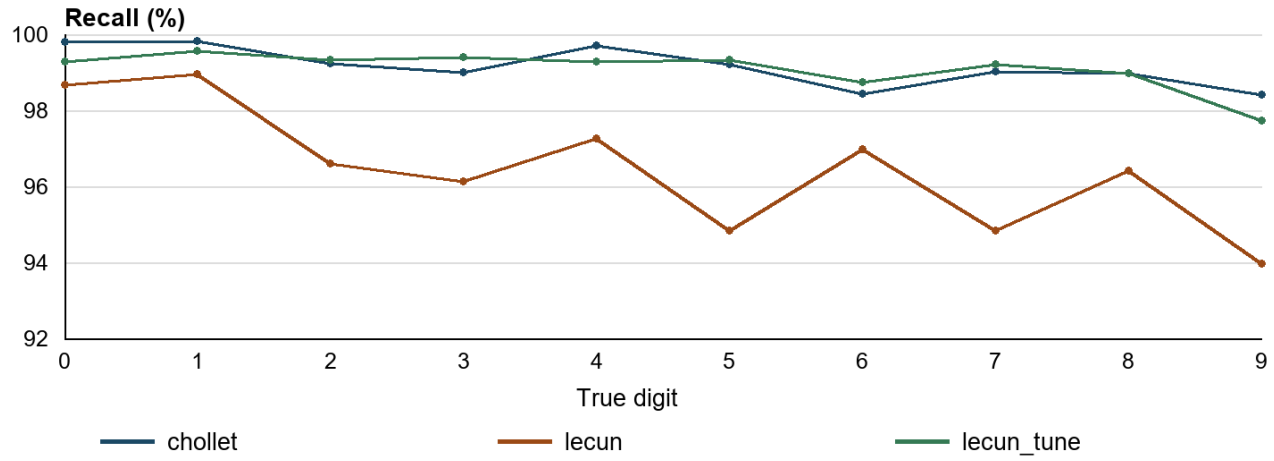
Source: Own illustration based on the recorded test predictions.

6.2 Comparison of the remaining errors

The misclassified examples reveal similar patterns of confusion across the three models, particularly between visually similar digits, including 4 and 6, 3 and 5, and 8 and 9. However, the equal number of displayed examples does not indicate equal performance. Across the full 10,000-image test set, Chollet produces 83 errors, LeCun produces 350, and LeCun Tuned produces 91.

Figure 6 compares the recall of each model across the ten-digit classes, providing a more detailed view of their classification performance. Additional graphs and confusion matrices are available in the accompanying Jupyter notebook for further examination.

Fig. 6: Recall by digit class for the three models



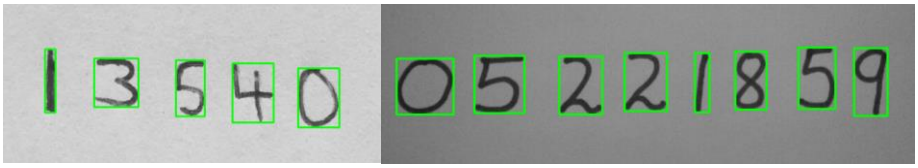
Source: Own illustration based on the recorded test results.

Digit 9 has the lowest recall for both LeCun (93.95%) and LeCun Tuned (97.72%). Nevertheless, the tuned model achieves higher recall for every digit class compared with the initial LeCun approximation. These results demonstrate that the improvements extend across all classes rather than being limited to a small number of digits. The complete confusion matrices and individual learning curves are retained in the saved experimental outputs.

6.3 Recognition of handwritten photographs

To demonstrate a practical application beyond the MNIST test set, the three trained models are applied to two photographs containing multiple handwritten digits. OpenCV is used to separate the handwritten content from the background, identify individual digit regions, and prepare each digit as a centred 28 × 28 image. The extracted digits are then processed and classified by each model using its corresponding preprocessing procedure.

Fig. 7: Detected handwritten digit regions in the two input photographs



Source: Own illustration based on the processed input photographs.

Tab. 4: Predicted digit sequences and mean confidence scores for the handwritten photographs

Image	Model	Prediction	Mean confidence
1	chollet	13540	99.99%
1	lecun	13540	95.49%
1	lecun_tune	13540	99.89%
2	chollet	05221859	99.85%
2	lecun	05221859	96.81%
2	lecun_tune	05221859	99.90%

Source: Own representation based on the recorded recognition results.

All three models predict the same digit sequences for both photographs. For the first photograph, Chollet produces the highest mean confidence of 99.99%, followed by LeCun Tuned at 99.89% and LeCun at 95.49%. For the second photograph, LeCun Tuned achieves the highest score of 99.90%, slightly exceeding Chollet at 99.85%, while LeCun reaches 96.81%.

Mean confidence is calculated as the average of the highest softmax scores across the extracted digits. Although these results demonstrate that all three models can be applied to segmented handwritten photographs, confidence scores do not necessarily reflect the correctness or reliability of predictions. Furthermore, two photographs provide only a limited demonstration and are insufficient to establish generalisation across different handwriting styles, backgrounds, or lighting conditions.

7 Reflection and conclusion

This project investigated whether a compact approximation of LeCun's historical convolutional network could achieve competitive handwritten digit recognition performance compared with a modern Chollet baseline. The initial LeCun approximation achieved 96.50% test accuracy with 3,246 trainable parameters, compared with Chollet's 99.17% accuracy and 34,826 parameters. Although the approximation required considerably fewer parameters and less training time, its lower recognition accuracy demonstrated the limitations of the initial configuration.

Hyperparameter tuning substantially improved the historical approximation. By preserving the original 28 × 28 image resolution, increasing the convolutional feature maps to 8 and 24, and introducing small random translations during training, LeCun Tuned achieved 99.09% test accuracy with 8,882 trainable parameters. This reduced the number of incorrect predictions from 350 to 91 compared

with the initial approximation. Relative to Chollet, the tuned model required approximately 74.5% fewer parameters while achieving an accuracy only 0.08 percentage points lower.

However, reduced model complexity did not translate into shorter training time. Although LeCun Tuned contained substantially fewer parameters than Chollet, its selected training run required 170.80 seconds compared with 143.47 seconds for Chollet. Furthermore, identifying the selected configuration required ten tuning trials with a combined fitting time of approximately 16.45 minutes. The improvement therefore demonstrates a reduction in model size rather than a reduction in the computational effort required during development.

The application to handwritten photographs demonstrated how OpenCV-based digit segmentation could be combined with the trained classifiers. All three models produced identical digit sequences for the two photographs, although their confidence scores differed. These observations illustrate the practical application of the models but cannot establish their reliability under a wider range of handwriting styles, lighting conditions, or image backgrounds.

Several limitations should be considered. The models were evaluated using a single fixed data split and training seed, and their preprocessing procedures differed. Consequently, the results reflect the performance of the complete model configurations rather than the isolated effects of individual architectural decisions. Additional experiments using different random seeds, independently collected handwriting samples, and more systematic hyperparameter comparisons would provide stronger evidence of generalisation and performance consistency.

Overall, Chollet achieved the highest observed test accuracy, while LeCun Tuned approached its performance with substantially fewer trainable parameters. The findings demonstrate that a compact architecture inspired by early convolutional networks can remain effective when its input representation, model capacity, and training configuration are appropriately adjusted. The tuned approximation therefore represents a useful alternative when model size is an important consideration.

References

Chollet, F. (2020). *Simple MNIST convnet*. Keras. https://keras.io/examples/vision/mnist_convnet/

LeCun, Y., Boser, B., Denker, J. S., Henderson, D., Howard, R. E., Hubbard, W., & Jackel, L. D. (1989). Handwritten digit recognition with a back-propagation network. In D. S. Touretzky (Ed.), *Advances in neural information processing systems* (Vol. 2, pp. 396–404). Morgan Kaufmann. https://proceedings.neurips.cc/paper_files/paper/1989/file/53c3bce66e43be4f209556518c2fcb54-Paper.pdf

LeCun, Y., Cortes, C., & Burges, C. J. C. (1998). *The MNIST database of handwritten digits* [Data set]. http://yann.lecun.com/exdb/mnist/

Appendix A Reproduction and evidence

Evidence files

The project includes the Jupyter notebook, trained models, recorded experimental results, and supporting output files. The comparison.csv file contains the test losses, accuracies, parameter counts, and training times of the three models.

The lecun_tune directory contains the hyperparameter search results, including trials.csv, best_config.json, the saved trial configurations, and their training histories. Individual model directories contain saved models, learning curves, confusion matrices, and examples of incorrect predictions.

The recognition/predictions.csv file records the digit sequences and confidence scores obtained from the two handwritten photographs.

Reproduction procedure

The project was developed using Python and the TensorFlow/Keras framework, with OpenCV for image processing, NumPy and pandas for data handling, and Matplotlib for visualisation. The required dependencies and supported version ranges are provided in requirements.txt.

To reproduce the saved evaluations, install the dependencies, open the Jupyter notebook, and execute its cells in order. The default configuration uses TRAIN_NEW = False to reuse previously trained models. To retrain the models, set TRAIN_NEW = True and select a different RUN_NAME to avoid overwriting existing results. A new TUNE_NAME can also be specified for an independent hyperparameter search.

The accompanying notebook PDF documents a second training execution, resulting in different training times from those presented in Table 3. However, the test results remain consistent, and these differences do not affect the conclusions.

The complete implementation, trained models, and supporting experimental results are available in the project repository:

https://github.com/NasiriMohsen/Handwritten-Digit-Recognition-with-Convolutional-Neural-Networks

1- Libraries

In [42]:
```
# System libraries
import os
import platform
import time
from pathlib import Path
```

In [43]:
```
# Data libraries
import numpy as np
import pandas as pd
import json
```

In [44]:
```
# Redirecting caches and adjusting log level
os.environ.setdefault('KERAS_HOME', str(Path.cwd() / '.cache' / 'keras'))
os.environ.setdefault('MPLCONFIGDIR', str(Path.cwd() / '.cache' / 'matplotlib'))
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "3"
```

In [45]:
```
# Machine Learning and Computer Vision Libraries
import cv2
import tensorflow as tf
tf.get_logger().setLevel("ERROR")
import keras
from keras import layers
```

In [46]:
```
# Graph libraries
from IPython.display import display, Image, Markdown
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
```

2- Variables

In [47]:
```
# Model training settings
TRAIN_NEW = True  # Change to retrain model
SEED = 42
EPOCHS = 15
BATCH_SIZE = 128
LEARNING_RATE = 0.001
```

In [48]:
```
# Model
BASE_MODEL_NAMES = ['chollet', 'lecun']
MODEL_NAMES = ['chollet', 'lecun', 'lecun_tune']
BASELINE_DROPOUT = 0.5
LECUN_DROPOUT = 0.2
```

In [49]:
```
# Dataset
TRAIN_COUNT = 54000
```

```
VALIDATION_COUNT = 6000
TEST_COUNT = 10000
IMAGE_SIZE = 28
LECUN_ACTIVE_SIZE = 16
LECUN_BORDER = (IMAGE_SIZE - LECUN_ACTIVE_SIZE) // 2
NUM_CLASSES = 10
```

In [50]:
```
# Path and files
ROOT = Path.cwd()
RUN_NAME = 'full'
RUN_DIR = ROOT / 'results' / RUN_NAME
MODEL_DIRS = {name: RUN_DIR / name for name in MODEL_NAMES}
TEST_IMAGES = [ROOT / 'Test Image 1.jpg', ROOT / 'Test Image 2.jpg']
RECOGNITION_DIR = RUN_DIR / 'recognition'
```

In [ ]:
```
# Hyperparameter tunning settings
TUNE_NAME = 'search_v2'
TUNE_DIR = MODEL_DIRS['lecun_tune']
TRIALS_DIR = TUNE_DIR / 'tuning' / TUNE_NAME
LEGACY_DIR = TUNE_DIR / 'tuning' / 'search_v1'
TUNE_BUDGET_MINUTES = 25
TUNE_PATIENCE = 6
TUNE_DEFAULTS = dict(active_size=28, filters=[4, 12], activation='tanh',
pooling='average', dropout=0.1, learning_rate=0.001, batch_size=128,
max_epochs=30, translation=0.0, reduce_lr=True)
TUNING_CONFIGS = [
    dict(TUNE_DEFAULTS, trial_id='small_long', learning_rate=0.003),
    dict(TUNE_DEFAULTS, trial_id='small_batch64', batch_size=64, dropout=0.05),
    dict(TUNE_DEFAULTS, trial_id='wide_tanh', filters=[8, 24]),
    dict(TUNE_DEFAULTS, trial_id='wide_relu', filters=[8, 24], activation='relu'),
    dict(TUNE_DEFAULTS, trial_id='wide_shift', filters=[8, 24], translation=0.07),
    dict(TUNE_DEFAULTS, trial_id='wide_relu_max', filters=[8, 24],
activation='relu',
         pooling='max', dropout=0.2, translation=0.07),
]
# Completed v1 values for documentation:
# size=16, dropout=0.2, lr=0.001, 15 epochs
# size=16, dropout=0.0, lr=0.001, 15 epochs
# size=28, dropout=0.0, lr=0.001, 15 epochs
# size=28, dropout=0.1, lr=0.003, 15 epochs
```

3- Functions

In [52]:
```
# Models: chollet, lecun, lecun_tune
def build_model(name, config=None):
    learning_rate = LEARNING_RATE
    if name == 'chollet':
        stack = [                                       # Based on Chollet's
baseline:
            layers.Conv2D(32, 3, activation='relu'),    # 32 filters (local
features)
            layers.MaxPooling2D(2),                     # Halve spatial size
keeping important features (pooling stage one)
```

```
            layers.Conv2D(64, 3, activation='relu'),    # 64 filters (complex
features)
            layers.MaxPooling2D(2),                     # Halve spatial size
keeping important features (second pooling stage)
            layers.Flatten(),                           # Converting spatial
features into a vector
            layers.Dropout(BASELINE_DROPOUT),           # Regularization to avoid
overfitting
        ]
    elif name == 'lecun':
        stack = [                                               # Historical
approximation:
            layers.Conv2D(4, 5, activation='tanh', name='H1'),  # Four 5x5 H1 maps
            layers.AveragePooling2D(2, name='H2'),              # Fixed H2-style
averaging pooling
            layers.SpatialDropout2D(LECUN_DROPOUT),             # Regularization
(not an exact sparse connection mask)
            layers.Conv2D(12, 5, activation='tanh', name='H3'), # Twelve 5x5 H3
maps
            layers.AveragePooling2D(2, name='H4'),              # Fixed H4-style
average pooling
            layers.Flatten(),                                   # Connecting
spatial features to the output
        ]
    elif name == 'lecun_tune':
        if config is None:
            raise ValueError('lecun_tune requires a trial configuration')
        learning_rate = config['learning_rate']
        pool = layers.AveragePooling2D if config['pooling'] == 'average' else
layers.MaxPooling2D
        stack = []
        if config['translation']:
            stack.append(layers.RandomTranslation(config['translation'],
                                config['translation'], fill_mode='constant',
fill_value=-1.0, seed=SEED))  # Small training-only shifts
        stack += [
# Historical approximation:
            layers.Conv2D(config['filters'][0], 5,
activation=config['activation'], name='H1'), # Testing more feature capacity and
tanh vs ReLU
            pool(2, name='H2'),
# Testing averaging vs strongest-response pooling
            layers.SpatialDropout2D(config['dropout']),
# Tunning regularization
            layers.Conv2D(config['filters'][1], 5,
activation=config['activation'], name='H3'), # Testing more feature capacity and
tanh vs ReLU
            pool(2, name='H4'),
# Testing averaging vs strongest-response pooling (same type as H2)
            layers.Flatten(),
# Connecting spatial features to the output
        ]
    else:
        raise ValueError(name)
    model = keras.Sequential([
        keras.Input((IMAGE_SIZE, IMAGE_SIZE, 1)),  # Input is 28x28 grayscale
```

```
image
        *stack,
        layers.Dense(NUM_CLASSES, activation='softmax'),  # Ten probabilities for
10 digits
    ], name=name)
    model.compile(optimizer=keras.optimizers.Adam(learning_rate=learning_rate),
loss='categorical_crossentropy', metrics=['accuracy'])
    return model
```

In [53]:
```
# Data/image processor
def preprocess(x, name):
    if name == 'lecun_tune':
        return preprocess_trial(x, BEST_CONFIG)
    x = x.astype('float32')[..., None] / 255.0
    if name == 'lecun':
        # Approximate historical image with a larger background
        x = tf.image.resize(x, (LECUN_ACTIVE_SIZE, LECUN_ACTIVE_SIZE),
method='bilinear').numpy()
        x = np.pad(x, ((0, 0), (LECUN_BORDER, LECUN_BORDER), (LECUN_BORDER,
LECUN_BORDER), (0, 0)))
        x = 2 * x - 1
    return x
```

In [54]:
```
# Model trainer
def train_model(name, xa, ya, xv, yv, xt, yt):
    keras.backend.clear_session()
    keras.utils.set_random_seed(SEED)
    model = build_model(name)
    model.summary()
    lines = []
    model.summary(print_fn=lambda s, **kwargs: lines.append(s))
    MODEL_DIRS[name].mkdir(parents=True, exist_ok=True)
    (MODEL_DIRS[name] / f'{name}_summary.txt').write_text('\n'.join(lines),
encoding='utf-8')
    train_x, val_x, test_x = [preprocess(z, name) for z in [xa, xv, xt]]
    train_y, val_y, test_y = [keras.utils.to_categorical(z, NUM_CLASSES) for z in
[ya, yv, yt]]
    # Measuring training and validation time, excluding image preparation and
final testing
    start = time.perf_counter()
    history = model.fit(train_x, train_y, validation_data=(val_x, val_y),
epochs=EPOCHS, batch_size=BATCH_SIZE, shuffle=True, verbose=2)
    seconds = time.perf_counter() - start
    loss, accuracy = model.evaluate(test_x, test_y, verbose=0)
    pred = model.predict(test_x, verbose=0).argmax(axis=1)
    model.save(MODEL_DIRS[name] / f'{name}.keras')
    (MODEL_DIRS[name] /
f'{name}_history.json').write_text(json.dumps(history.history, indent=2))
    figures(MODEL_DIRS[name], name, history.history, yt, pred, xt)
    return dict(model=name, test_loss=loss, test_accuracy=accuracy,
training_seconds=seconds, parameters=model.count_params(),
epochs=EPOCHS,train_samples=len(xa), validation_samples=len(xv),
test_samples=len(xt), seed=SEED)
```

In [55]:
```
# Figure saver: learning curve, confusion matrix, wrong predictions
def figures(out, name, history, truth, pred, raw):
    fig, axs = plt.subplots(1, 2, figsize=(10, 4))
    for ax, metric in zip(axs, ['accuracy', 'loss']):
        for key in [metric, 'val_' + metric]:
            ax.plot(range(1, len(history[key])+1), history[key], label=key)
        ax.set(xlabel='Epoch', ylabel=metric, title=name)
        ax.legend()
    fig.tight_layout(); fig.savefig(out / f'{name}_learning.png', dpi=160);
plt.close(fig)
    cm = np.zeros((10, 10), dtype=int)
    # Counting each prediction by actual digit and predicted digit
    np.add.at(cm, (truth, pred), 1)
    np.savetxt(out / f'{name}_confusion.csv', cm, fmt='%d', delimiter=',')
    fig, ax = plt.subplots(figsize=(7, 6))
    im = ax.imshow(cm, cmap='Blues')
    for i in range(10):
        for j in range(10):
            ax.text(j, i, str(cm[i,j]), ha='center', va='center', color='white' if
cm[i,j] > cm.max()/2 else 'black', fontsize=7)
    ax.set(xticks=range(10), yticks=range(10), xlabel='Predicted', ylabel='True',
title=name)
    fig.colorbar(im, ax=ax); fig.tight_layout()
    fig.savefig(out / f'{name}_confusion.png', dpi=160); plt.close(fig)
    wrong = np.flatnonzero(truth != pred)[:16]
    if len(wrong):
        fig, axs = plt.subplots(4, 4, figsize=(7, 7))
        for ax in axs.flat: ax.axis('off')
        for ax, idx in zip(axs.flat, wrong):
            ax.imshow(raw[idx], cmap='gray'); ax.set_title(f'True {truth[idx]} /
Pred {pred[idx]}')
        fig.tight_layout(); fig.savefig(out / f'{name}_errors.png', dpi=160);
plt.close(fig)
```

In [56]:
```
# Images digit extractor
def extract_digits(path):
    gray = cv2.imread(str(path), cv2.IMREAD_GRAYSCALE)
    if gray is None:
        raise FileNotFoundError(path)
    # Separating handwriting from the background while allowing for uneven
lighting
    binary = cv2.adaptiveThreshold(cv2.GaussianBlur(gray, (5, 5), 0), 255,
cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY_INV, 51, 12)
    _, _, stats, _ = cv2.connectedComponentsWithStats(binary)
    # Filtering small marks and large regions, sorting remaining digits from left
to right
    boxes = sorted([(x, y, w, h) for x, y, w, h, area in stats[1:] if h >
gray.shape[0] * 0.08 and area > 70 and w < gray.shape[1] * 0.3])
    digits = []
    for x, y, w, h in boxes:
        crop = binary[y:y+h, x:x+w]
        # Keeping the digit proportions and centering it inside the same size
canvas as MNIST
        scale = 20 / max(w, h)
        size = (max(1, round(w*scale)), max(1, round(h*scale)))
```

```
        resized = cv2.resize(crop, size)
        canvas = np.zeros((28, 28), dtype=np.uint8)
        rh, rw = resized.shape
        canvas[(28-rh)//2:(28-rh)//2+rh, (28-rw)//2:(28-rw)//2+rw] = resized
        digits.append(canvas)
    return np.array(digits), gray, boxes
```

In [57]:
```
# Data/image processor based on trial: resize, pad, normalise
def preprocess_trial(raw, config):
    images = raw.astype('float32')[..., None] / 255.0
    size = config['active_size']
    if size != IMAGE_SIZE:
        images = tf.image.resize(images, (size, size), method='bilinear').numpy()
        pad = (IMAGE_SIZE - size) // 2
        images = np.pad(images, ((0, 0), (pad, pad), (pad, pad), (0, 0)))
    return 2 * images - 1
```

In [58]:
```
# Hyperparameter grid search optimiser
class SearchDeadline(keras.callbacks.Callback):
    """Stop fitting near the wall-clock budget; early stopping restores best
weights."""
    def __init__(self, deadline, progress_path, trial_id):
        super().__init__()
        self.deadline, self.progress_path, self.trial_id = deadline,
progress_path, trial_id
        self.budget_stopped = False
    def on_train_batch_end(self, batch, logs=None):
        if time.perf_counter() >= self.deadline:
            self.model.stop_training = True
            self.budget_stopped = True
    def on_epoch_end(self, epoch, logs=None):
        self.progress_path.write_text(json.dumps(dict(trial=self.trial_id,
epoch=epoch+1,
            **{k: float(v) for k,v in (logs or {}).items()}), indent=2))
# Based on train and validation sets only
def tune_lecun(train_x, train_y, val_x, val_y):
    TRIALS_DIR.mkdir(parents=True, exist_ok=True)
    manifest = dict(configs=TUNING_CONFIGS, patience=TUNE_PATIENCE, seed=SEED,
train=len(train_x), validation=len(val_x), tensorflow=tf.__version__,
keras=keras.__version__, platform=platform.platform())
    manifest_path = TRIALS_DIR / 'search.json'
    # Checking saved settings to avoid mixing results from different experiments
    if manifest_path.exists() and json.loads(manifest_path.read_text()) !=
manifest:
        raise ValueError('Settings/environment changed; choose a new TUNE_NAME.')
    manifest_path.write_text(json.dumps(manifest, indent=2))
    start_search = time.perf_counter()
    deadline = start_search + TUNE_BUDGET_MINUTES * 60
    rows = []
    # Importing previous trials as competitors
    for record in sorted(LEGACY_DIR.glob('trial_*/result.json')):
        old = json.loads(record.read_text())
        config = dict(TUNE_DEFAULTS, active_size=old['active_size'],
dropout=old['dropout'], learning_rate=old['learning_rate'], max_epochs=15,
reduce_lr=False)
```

```
        rows.append(dict(trial_id='v1_'+record.parent.name, config=config,
            model_path=str((record.parent/'model.keras').relative_to(TUNE_DIR)),

history_path=str((record.parent/'history.json').relative_to(TUNE_DIR)),
            val_loss=old['val_loss'], val_accuracy=old['val_accuracy'],
            training_seconds=old['training_seconds'],
epochs_run=old['epochs_run'],
            best_epoch=old['best_epoch'], status='imported', parameters=3246))
    for config in TUNING_CONFIGS:
        trial_id = TUNE_NAME + '_' + config['trial_id']
        folder = TRIALS_DIR / config['trial_id']
        record = folder / 'result.json'
        # Reusing completed trials instead of training the same settings again
        if record.exists() and (folder/'model.keras').exists():
            saved = json.loads(record.read_text())
            if saved['config'] != config:
                raise ValueError('Trial changed; use a new TUNE_NAME.')
            rows.append(saved)
            continue
        if time.perf_counter() >= deadline:
            break
        folder.mkdir(parents=True, exist_ok=True)
        keras.backend.clear_session()
        keras.utils.set_random_seed(SEED)
        model = build_model('lecun_tune', config)
        tx, vx = preprocess_trial(train_x, config), preprocess_trial(val_x,
config)
        ty, vy = [keras.utils.to_categorical(z, NUM_CLASSES) for z in [train_y,
val_y]]
        limiter = SearchDeadline(deadline, TRIALS_DIR/'progress.json', trial_id)
        # Restoring the best validation weights and stopping when improvement
stalls
        callbacks = [keras.callbacks.EarlyStopping(monitor='val_loss',
patience=TUNE_PATIENCE, restore_best_weights=True), limiter,
keras.callbacks.CSVLogger(str(folder/'epochs.csv'))]
        # Taking smaller learning steps when validation loss stops improving
        if config['reduce_lr']:
            callbacks.append(keras.callbacks.ReduceLROnPlateau(monitor='val_loss',
factor=0.5, patience=2, min_delta=0.0001, min_lr=0.00005))
        print('Training', trial_id, config, flush=True)
        start = time.perf_counter()
        history = model.fit(tx, ty, validation_data=(vx, vy),
epochs=config['max_epochs'], batch_size=config['batch_size'], callbacks=callbacks,
verbose=2)
        seconds = time.perf_counter() - start
        loss, accuracy = model.evaluate(vx, vy, verbose=0)
        model.save(folder/'model.keras')
        (folder/'history.json').write_text(json.dumps(history.history, indent=2))
        lines=[]
        model.summary(print_fn=lambda s, **kw: lines.append(s))
        (folder/'summary.txt').write_text('\n'.join(lines), encoding='utf-8')
        row = dict(trial_id=trial_id, config=config,
            model_path=str((folder/'model.keras').relative_to(TUNE_DIR)),
            history_path=str((folder/'history.json').relative_to(TUNE_DIR)),
            val_loss=float(loss), val_accuracy=float(accuracy),
training_seconds=seconds,
```

```
            epochs_run=len(history.history['loss']),
best_epoch=int(np.argmin(history.history['val_loss']))+1,
            status='budget_stopped' if limiter.budget_stopped else 'complete',
parameters=model.count_params())
        record.write_text(json.dumps(row, indent=2))
        rows.append(row)
        pd.json_normalize(rows).to_csv(TRIALS_DIR/'leaderboard.csv', index=False)
    if not rows:
        raise RuntimeError('No completed trials available.')
    # Selecting by validation loss, test results do not influence the winner
    best = min(rows, key=lambda row: (row['val_loss'], row['trial_id']))
    table = pd.json_normalize(rows).sort_values(['val_loss',
'trial_id']).reset_index(drop=True)
    table.to_csv(TUNE_DIR/'trials.csv', index=False)
    table.to_csv(TRIALS_DIR/'leaderboard.csv', index=False)
    (TUNE_DIR/'best_config.json').write_text(json.dumps(best, indent=2))
    complete_ids = {r['trial_id'] for r in rows}
    pending = [c['trial_id'] for c in TUNING_CONFIGS if
TUNE_NAME+'_'+c['trial_id'] not in complete_ids]
    (TRIALS_DIR/'status.json').write_text(json.dumps(dict(pending=pending,
elapsed_seconds=time.perf_counter()-start_search,
budget_minutes=TUNE_BUDGET_MINUTES), indent=2))
    return best, table
```

4- Main

In [59]:
```
# Spliting Dataset (Train set: 54k, Validation set: 6k, Test set: 10k)
keras.utils.set_random_seed(SEED)
tf.config.experimental.enable_op_determinism()
(x, y), (xt, yt) = keras.datasets.mnist.load_data()
assert len(x) == TRAIN_COUNT + VALIDATION_COUNT and len(xt) == TEST_COUNT
xa, ya = x[:TRAIN_COUNT], y[:TRAIN_COUNT]
xv, yv = x[TRAIN_COUNT:], y[TRAIN_COUNT:]
print(f'Training: {len(xa):,}; validation: {len(xv):,}; test: {len(xt):,}')
```

```
Training: 54,000; validation: 6,000; test: 10,000
```

In [ ]:
```
# Creating or reusing model
RUN_DIR.mkdir(parents=True, exist_ok=True)
for folder in MODEL_DIRS.values():
    folder.mkdir(parents=True, exist_ok=True)
comparison_path = RUN_DIR / 'comparison.csv'
reuse = (comparison_path.exists() and not TRAIN_NEW and all((MODEL_DIRS[name] /
f'{name}.keras').exists() for name in BASE_MODEL_NAMES))
if not reuse and not TRAIN_NEW:
    raise FileNotFoundError('Saved baseline files are missing; check paths before
retraining or set the TRAIN_NEW flag to True')
if comparison_path.exists() and TRAIN_NEW:
    raise FileExistsError('Choose a new RUN_NAME before retraining.')
if reuse:
    results = pd.read_csv(comparison_path)
    results = results[results['model'].isin(BASE_MODEL_NAMES)]
    if set(results['model']) != set(BASE_MODEL_NAMES) or len(results) !=
len(BASE_MODEL_NAMES):
```

```
        raise ValueError('Saved run is incomplete. Choose a new RUN_NAME.')
    print('Reusing completed results')
else:
    metadata = dict(epochs=EPOCHS, batch_size=BATCH_SIZE, seed=SEED, smoke=False,
                    learning_rate=LEARNING_RATE, train=len(xa),
validation=len(xv), test=len(xt),
                    python=platform.python_version(), tensorflow=tf.__version__,
keras=keras.__version__,
                    platform=platform.platform(), devices=[str(d) for d in
tf.config.list_physical_devices()],
                    timing='perf_counter around fit; includes validation; excludes
preprocessing and evaluation')
    rows = []
if reuse:
    for name in BASE_MODEL_NAMES:
        model_path = MODEL_DIRS[name] / f'{name}.keras'
        loaded = keras.models.load_model(model_path)
        if loaded.name != name:
            before = loaded.get_weights()
            loaded.name = name
            loaded.save(model_path)
            reloaded = keras.models.load_model(model_path)
            for a, b in zip(before, reloaded.get_weights()):
                np.testing.assert_array_equal(a, b)
```

In [61]:
```
# Training Chollet baseline model
if not reuse:
    rows.append(train_model('chollet', xa, ya, xv, yv, xt, yt))
    pd.DataFrame(rows).to_csv(comparison_path, index=False)
```

Model: "chollet"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dropout (Dropout)	(None, 1600)	0
dense (Dense)	(None, 10)	16,010

Total params: 34,826 (136.04 KB)

Trainable params: 34,826 (136.04 KB)

Non-trainable params: 0 (0.00 B)

```
Epoch 1/15
422/422 - 11s - 26ms/step - accuracy: 0.8925 - loss: 0.3585 - val_accuracy: 0.9778 -
val_loss: 0.0805
Epoch 2/15
422/422 - 10s - 24ms/step - accuracy: 0.9660 - loss: 0.1096 - val_accuracy: 0.9832 -
val_loss: 0.0581
Epoch 3/15
422/422 - 10s - 25ms/step - accuracy: 0.9740 - loss: 0.0849 - val_accuracy: 0.9877 -
val_loss: 0.0461
Epoch 4/15
422/422 - 20s - 49ms/step - accuracy: 0.9781 - loss: 0.0701 - val_accuracy: 0.9878 -
val_loss: 0.0426
Epoch 5/15
422/422 - 10s - 23ms/step - accuracy: 0.9806 - loss: 0.0633 - val_accuracy: 0.9887 -
val_loss: 0.0383
Epoch 6/15
422/422 - 9s - 21ms/step - accuracy: 0.9825 - loss: 0.0563 - val_accuracy: 0.9893 -
val_loss: 0.0373
Epoch 7/15
422/422 - 9s - 21ms/step - accuracy: 0.9844 - loss: 0.0514 - val_accuracy: 0.9910 -
val_loss: 0.0342
Epoch 8/15
422/422 - 9s - 21ms/step - accuracy: 0.9843 - loss: 0.0495 - val_accuracy: 0.9910 -
val_loss: 0.0326
Epoch 9/15
422/422 - 9s - 20ms/step - accuracy: 0.9851 - loss: 0.0463 - val_accuracy: 0.9907 -
val_loss: 0.0316
Epoch 10/15
422/422 - 9s - 20ms/step - accuracy: 0.9865 - loss: 0.0439 - val_accuracy: 0.9913 -
val_loss: 0.0304
Epoch 11/15
422/422 - 9s - 21ms/step - accuracy: 0.9871 - loss: 0.0410 - val_accuracy: 0.9920 -
val_loss: 0.0302
Epoch 12/15
422/422 - 10s - 24ms/step - accuracy: 0.9878 - loss: 0.0384 - val_accuracy: 0.9915 -
val_loss: 0.0287
Epoch 13/15
422/422 - 11s - 26ms/step - accuracy: 0.9881 - loss: 0.0370 - val_accuracy: 0.9925 -
val_loss: 0.0269
Epoch 14/15
422/422 - 10s - 23ms/step - accuracy: 0.9888 - loss: 0.0350 - val_accuracy: 0.9927 -
val_loss: 0.0274
Epoch 15/15
422/422 - 11s - 25ms/step - accuracy: 0.9893 - loss: 0.0336 - val_accuracy: 0.9925 -
val_loss: 0.0278
```

In [62]:
```python
# Training Historical approximation model
if not reuse:
    rows.append(train_model('lecun', xa, ya, xv, yv, xt, yt))
    pd.DataFrame(rows).to_csv(comparison_path, index=False)
```

```
Model: "lecun"
```

Layer (type)	Output Shape	Param #
H1 (Conv2D)	(None, 24, 24, 4)	104
H2 (AveragePooling2D)	(None, 12, 12, 4)	0
spatial_dropout2d (SpatialDropout2D)	(None, 12, 12, 4)	0
H3 (Conv2D)	(None, 8, 8, 12)	1,212
H4 (AveragePooling2D)	(None, 4, 4, 12)	0
flatten (Flatten)	(None, 192)	0
dense (Dense)	(None, 10)	1,930

Total params: 3,246 (12.68 KB)

Trainable params: 3,246 (12.68 KB)

Non-trainable params: 0 (0.00 B)

```
Epoch 1/15
422/422 - 8s - 18ms/step - accuracy: 0.7427 - loss: 0.9128 - val_accuracy: 0.9013 -
val_loss: 0.3589
Epoch 2/15
422/422 - 3s - 8ms/step - accuracy: 0.8851 - loss: 0.3898 - val_accuracy: 0.9263 -
val_loss: 0.2612
Epoch 3/15
422/422 - 3s - 7ms/step - accuracy: 0.9080 - loss: 0.3102 - val_accuracy: 0.9385 -
val_loss: 0.2170
Epoch 4/15
422/422 - 3s - 8ms/step - accuracy: 0.9209 - loss: 0.2644 - val_accuracy: 0.9453 -
val_loss: 0.1883
Epoch 5/15
422/422 - 4s - 8ms/step - accuracy: 0.9306 - loss: 0.2325 - val_accuracy: 0.9522 -
val_loss: 0.1682
Epoch 6/15
422/422 - 4s - 10ms/step - accuracy: 0.9372 - loss: 0.2116 - val_accuracy: 0.9562 -
val_loss: 0.1552
Epoch 7/15
422/422 - 3s - 8ms/step - accuracy: 0.9422 - loss: 0.1927 - val_accuracy: 0.9590 -
val_loss: 0.1424
Epoch 8/15
422/422 - 3s - 8ms/step - accuracy: 0.9459 - loss: 0.1802 - val_accuracy: 0.9597 -
val_loss: 0.1349
Epoch 9/15
422/422 - 3s - 8ms/step - accuracy: 0.9489 - loss: 0.1691 - val_accuracy: 0.9622 -
val_loss: 0.1251
Epoch 10/15
422/422 - 3s - 7ms/step - accuracy: 0.9512 - loss: 0.1614 - val_accuracy: 0.9647 -
val_loss: 0.1203
Epoch 11/15
422/422 - 3s - 7ms/step - accuracy: 0.9538 - loss: 0.1519 - val_accuracy: 0.9652 -
val_loss: 0.1160
Epoch 12/15
422/422 - 3s - 7ms/step - accuracy: 0.9554 - loss: 0.1469 - val_accuracy: 0.9663 -
val_loss: 0.1096
Epoch 13/15
422/422 - 3s - 7ms/step - accuracy: 0.9569 - loss: 0.1404 - val_accuracy: 0.9672 -
val_loss: 0.1071
Epoch 14/15
422/422 - 3s - 7ms/step - accuracy: 0.9594 - loss: 0.1350 - val_accuracy: 0.9688 -
val_loss: 0.1021
Epoch 15/15
422/422 - 3s - 7ms/step - accuracy: 0.9596 - loss: 0.1306 - val_accuracy: 0.9687 -
val_loss: 0.0988
```

In [63]:

```python
# Tunning Historical approximation model
BEST_TRIAL, tuning_results = tune_lecun(xa, ya, xv, yv)
BEST_CONFIG = BEST_TRIAL['config']
tuned_model = keras.models.load_model(TUNE_DIR / BEST_TRIAL['model_path'])
tuned_model.name = 'lecun_tune'
tuned_model.save(TUNE_DIR / 'lecun_tune.keras')
lines=[]
with __import__("IPython.utils.capture", fromlist=
["capture_output"]).capture_output():
    tuned_model.summary(print_fn=lambda s, **kw: lines.append(s))
```

```
(TUNE_DIR/'lecun_tune_summary.txt').write_text('\n'.join(lines), encoding='utf-8')
evaluation_dir = TUNE_DIR / 'evaluations'
evaluation_dir.mkdir(exist_ok=True)
evaluation_path = evaluation_dir / (BEST_TRIAL['trial_id']+'.json')
history = json.loads((TUNE_DIR / BEST_TRIAL['history_path']).read_text())
# Applying the winning image preparation before evaluating on the separate test
set
test_x = preprocess_trial(xt, BEST_CONFIG)
loss, accuracy = tuned_model.evaluate(test_x, keras.utils.to_categorical(yt,
NUM_CLASSES), verbose=0)
tuned_result = dict(model='lecun_tune', test_loss=float(loss),
test_accuracy=float(accuracy),
    winning_trial_seconds=BEST_TRIAL['training_seconds'],
parameters=tuned_model.count_params(),
    epochs=BEST_TRIAL['epochs_run'], selected_epoch=BEST_TRIAL['best_epoch'],
    train_samples=len(xa), validation_samples=len(xv), test_samples=len(xt),
seed=SEED,
    selected_trial=BEST_TRIAL['trial_id'])
evaluation_path.write_text(json.dumps(tuned_result, indent=2))
# Including the cost of all trials, not only the winning model
tuned_result['training_seconds'] = float(tuning_results['training_seconds'].sum())
(TUNE_DIR/'evaluation.json').write_text(json.dumps(tuned_result, indent=2))
(TUNE_DIR/'lecun_tune_history.json').write_text(json.dumps(history, indent=2))
prediction = tuned_model.predict(test_x, verbose=0).argmax(axis=1)
figures(TUNE_DIR, 'lecun_tune', history, yt, prediction, xt)
print('Selected on validation loss:', BEST_TRIAL['trial_id'], BEST_CONFIG)
print('Loaded selected trial:', BEST_TRIAL['trial_id'])
print(f"Verified test accuracy: {tuned_result['test_accuracy']:.2%}")
```

```
Training search_v2_small_long {'active_size': 28, 'filters': [4, 12], 'activation':
'tanh', 'pooling': 'average', 'dropout': 0.1, 'learning_rate': 0.003, 'batch_size':
128, 'max_epochs': 30, 'translation': 0.0, 'reduce_lr': True, 'trial_id':
'small_long'}
Epoch 1/30
422/422 - 4s - 10ms/step - accuracy: 0.8808 - loss: 0.4032 - val_accuracy: 0.9605 -
val_loss: 0.1488 - learning_rate: 0.0030
Epoch 2/30
422/422 - 3s - 7ms/step - accuracy: 0.9525 - loss: 0.1612 - val_accuracy: 0.9743 -
val_loss: 0.0985 - learning_rate: 0.0030
Epoch 3/30
422/422 - 3s - 7ms/step - accuracy: 0.9654 - loss: 0.1177 - val_accuracy: 0.9787 -
val_loss: 0.0790 - learning_rate: 0.0030
Epoch 4/30
422/422 - 3s - 7ms/step - accuracy: 0.9715 - loss: 0.0974 - val_accuracy: 0.9812 -
val_loss: 0.0695 - learning_rate: 0.0030
Epoch 5/30
422/422 - 3s - 6ms/step - accuracy: 0.9744 - loss: 0.0852 - val_accuracy: 0.9832 -
val_loss: 0.0621 - learning_rate: 0.0030
Epoch 6/30
422/422 - 3s - 6ms/step - accuracy: 0.9766 - loss: 0.0776 - val_accuracy: 0.9843 -
val_loss: 0.0581 - learning_rate: 0.0030
Epoch 7/30
422/422 - 3s - 7ms/step - accuracy: 0.9784 - loss: 0.0711 - val_accuracy: 0.9847 -
val_loss: 0.0555 - learning_rate: 0.0030
Epoch 8/30
422/422 - 3s - 7ms/step - accuracy: 0.9796 - loss: 0.0673 - val_accuracy: 0.9863 -
val_loss: 0.0520 - learning_rate: 0.0030
Epoch 9/30
422/422 - 3s - 6ms/step - accuracy: 0.9813 - loss: 0.0624 - val_accuracy: 0.9850 -
val_loss: 0.0510 - learning_rate: 0.0030
Epoch 10/30
422/422 - 3s - 7ms/step - accuracy: 0.9820 - loss: 0.0604 - val_accuracy: 0.9857 -
val_loss: 0.0501 - learning_rate: 0.0030
Epoch 11/30
422/422 - 3s - 6ms/step - accuracy: 0.9822 - loss: 0.0576 - val_accuracy: 0.9862 -
val_loss: 0.0493 - learning_rate: 0.0030
Epoch 12/30
422/422 - 3s - 7ms/step - accuracy: 0.9825 - loss: 0.0564 - val_accuracy: 0.9860 -
val_loss: 0.0502 - learning_rate: 0.0030
Epoch 13/30
422/422 - 3s - 7ms/step - accuracy: 0.9835 - loss: 0.0536 - val_accuracy: 0.9863 -
val_loss: 0.0470 - learning_rate: 0.0030
Epoch 14/30
422/422 - 3s - 7ms/step - accuracy: 0.9841 - loss: 0.0520 - val_accuracy: 0.9875 -
val_loss: 0.0467 - learning_rate: 0.0030
Epoch 15/30
422/422 - 3s - 7ms/step - accuracy: 0.9844 - loss: 0.0497 - val_accuracy: 0.9862 -
val_loss: 0.0470 - learning_rate: 0.0030
Epoch 16/30
422/422 - 3s - 7ms/step - accuracy: 0.9845 - loss: 0.0489 - val_accuracy: 0.9867 -
val_loss: 0.0478 - learning_rate: 0.0030
Epoch 17/30
422/422 - 3s - 7ms/step - accuracy: 0.9869 - loss: 0.0432 - val_accuracy: 0.9878 -
val_loss: 0.0428 - learning_rate: 0.0015
Epoch 18/30
```

```
422/422 - 3s - 6ms/step - accuracy: 0.9872 - loss: 0.0424 - val_accuracy: 0.9887 -
val_loss: 0.0419 - learning_rate: 0.0015
Epoch 19/30
422/422 - 3s - 6ms/step - accuracy: 0.9877 - loss: 0.0419 - val_accuracy: 0.9878 -
val_loss: 0.0419 - learning_rate: 0.0015
Epoch 20/30
422/422 - 3s - 7ms/step - accuracy: 0.9874 - loss: 0.0411 - val_accuracy: 0.9873 -
val_loss: 0.0420 - learning_rate: 0.0015
Epoch 21/30
422/422 - 3s - 7ms/step - accuracy: 0.9882 - loss: 0.0395 - val_accuracy: 0.9878 -
val_loss: 0.0419 - learning_rate: 7.5000e-04
Epoch 22/30
422/422 - 3s - 6ms/step - accuracy: 0.9885 - loss: 0.0383 - val_accuracy: 0.9882 -
val_loss: 0.0412 - learning_rate: 7.5000e-04
Epoch 23/30
422/422 - 3s - 7ms/step - accuracy: 0.9885 - loss: 0.0384 - val_accuracy: 0.9880 -
val_loss: 0.0411 - learning_rate: 7.5000e-04
Epoch 24/30
422/422 - 3s - 7ms/step - accuracy: 0.9888 - loss: 0.0371 - val_accuracy: 0.9878 -
val_loss: 0.0411 - learning_rate: 7.5000e-04
Epoch 25/30
422/422 - 3s - 7ms/step - accuracy: 0.9887 - loss: 0.0373 - val_accuracy: 0.9878 -
val_loss: 0.0412 - learning_rate: 7.5000e-04
Epoch 26/30
422/422 - 3s - 7ms/step - accuracy: 0.9891 - loss: 0.0358 - val_accuracy: 0.9870 -
val_loss: 0.0418 - learning_rate: 3.7500e-04
Epoch 27/30
422/422 - 3s - 6ms/step - accuracy: 0.9891 - loss: 0.0366 - val_accuracy: 0.9885 -
val_loss: 0.0408 - learning_rate: 3.7500e-04
Epoch 28/30
422/422 - 3s - 7ms/step - accuracy: 0.9892 - loss: 0.0357 - val_accuracy: 0.9877 -
val_loss: 0.0409 - learning_rate: 3.7500e-04
Epoch 29/30
422/422 - 3s - 7ms/step - accuracy: 0.9895 - loss: 0.0350 - val_accuracy: 0.9878 -
val_loss: 0.0410 - learning_rate: 3.7500e-04
Epoch 30/30
422/422 - 3s - 6ms/step - accuracy: 0.9891 - loss: 0.0357 - val_accuracy: 0.9880 -
val_loss: 0.0404 - learning_rate: 1.8750e-04
```

```
Training search_v2_small_batch64 {'active_size': 28, 'filters': [4, 12],
'activation': 'tanh', 'pooling': 'average', 'dropout': 0.05, 'learning_rate': 0.001,
'batch_size': 64, 'max_epochs': 30, 'translation': 0.0, 'reduce_lr': True,
'trial_id': 'small_batch64'}
Epoch 1/30
844/844 - 6s - 7ms/step - accuracy: 0.8546 - loss: 0.5012 - val_accuracy: 0.9423 -
val_loss: 0.2056 - learning_rate: 0.0010
Epoch 2/30
844/844 - 4s - 5ms/step - accuracy: 0.9348 - loss: 0.2210 - val_accuracy: 0.9642 -
val_loss: 0.1395 - learning_rate: 0.0010
Epoch 3/30
844/844 - 4s - 5ms/step - accuracy: 0.9541 - loss: 0.1599 - val_accuracy: 0.9713 -
val_loss: 0.1081 - learning_rate: 0.0010
Epoch 4/30
844/844 - 4s - 5ms/step - accuracy: 0.9624 - loss: 0.1289 - val_accuracy: 0.9757 -
val_loss: 0.0905 - learning_rate: 0.0010
Epoch 5/30
844/844 - 4s - 5ms/step - accuracy: 0.9681 - loss: 0.1095 - val_accuracy: 0.9788 -
val_loss: 0.0799 - learning_rate: 0.0010
Epoch 6/30
844/844 - 4s - 5ms/step - accuracy: 0.9714 - loss: 0.0968 - val_accuracy: 0.9823 -
val_loss: 0.0719 - learning_rate: 0.0010
Epoch 7/30
844/844 - 4s - 5ms/step - accuracy: 0.9738 - loss: 0.0884 - val_accuracy: 0.9832 -
val_loss: 0.0670 - learning_rate: 0.0010
Epoch 8/30
844/844 - 4s - 5ms/step - accuracy: 0.9763 - loss: 0.0809 - val_accuracy: 0.9840 -
val_loss: 0.0627 - learning_rate: 0.0010
Epoch 9/30
844/844 - 4s - 5ms/step - accuracy: 0.9780 - loss: 0.0754 - val_accuracy: 0.9845 -
val_loss: 0.0595 - learning_rate: 0.0010
Epoch 10/30
844/844 - 4s - 5ms/step - accuracy: 0.9791 - loss: 0.0705 - val_accuracy: 0.9852 -
val_loss: 0.0571 - learning_rate: 0.0010
Epoch 11/30
844/844 - 4s - 5ms/step - accuracy: 0.9805 - loss: 0.0664 - val_accuracy: 0.9855 -
val_loss: 0.0543 - learning_rate: 0.0010
Epoch 12/30
844/844 - 4s - 5ms/step - accuracy: 0.9816 - loss: 0.0629 - val_accuracy: 0.9857 -
val_loss: 0.0524 - learning_rate: 0.0010
Epoch 13/30
844/844 - 4s - 5ms/step - accuracy: 0.9821 - loss: 0.0610 - val_accuracy: 0.9858 -
val_loss: 0.0514 - learning_rate: 0.0010
Epoch 14/30
844/844 - 4s - 5ms/step - accuracy: 0.9827 - loss: 0.0584 - val_accuracy: 0.9870 -
val_loss: 0.0492 - learning_rate: 0.0010
Epoch 15/30
844/844 - 4s - 5ms/step - accuracy: 0.9836 - loss: 0.0561 - val_accuracy: 0.9865 -
val_loss: 0.0485 - learning_rate: 0.0010
Epoch 16/30
844/844 - 4s - 5ms/step - accuracy: 0.9845 - loss: 0.0535 - val_accuracy: 0.9872 -
val_loss: 0.0483 - learning_rate: 0.0010
Epoch 17/30
844/844 - 4s - 5ms/step - accuracy: 0.9844 - loss: 0.0522 - val_accuracy: 0.9873 -
val_loss: 0.0472 - learning_rate: 0.0010
Epoch 18/30
```

```
844/844 - 4s - 5ms/step - accuracy: 0.9849 - loss: 0.0507 - val_accuracy: 0.9882 -
val_loss: 0.0465 - learning_rate: 0.0010
Epoch 19/30
844/844 - 4s - 5ms/step - accuracy: 0.9852 - loss: 0.0491 - val_accuracy: 0.9882 -
val_loss: 0.0453 - learning_rate: 0.0010
Epoch 20/30
844/844 - 4s - 5ms/step - accuracy: 0.9862 - loss: 0.0471 - val_accuracy: 0.9878 -
val_loss: 0.0449 - learning_rate: 0.0010
Epoch 21/30
844/844 - 4s - 5ms/step - accuracy: 0.9861 - loss: 0.0463 - val_accuracy: 0.9887 -
val_loss: 0.0430 - learning_rate: 0.0010
Epoch 22/30
844/844 - 4s - 5ms/step - accuracy: 0.9866 - loss: 0.0458 - val_accuracy: 0.9885 -
val_loss: 0.0440 - learning_rate: 0.0010
Epoch 23/30
844/844 - 4s - 5ms/step - accuracy: 0.9868 - loss: 0.0439 - val_accuracy: 0.9885 -
val_loss: 0.0429 - learning_rate: 0.0010
Epoch 24/30
844/844 - 4s - 5ms/step - accuracy: 0.9878 - loss: 0.0414 - val_accuracy: 0.9897 -
val_loss: 0.0420 - learning_rate: 5.0000e-04
Epoch 25/30
844/844 - 4s - 5ms/step - accuracy: 0.9879 - loss: 0.0407 - val_accuracy: 0.9895 -
val_loss: 0.0424 - learning_rate: 5.0000e-04
Epoch 26/30
844/844 - 4s - 5ms/step - accuracy: 0.9879 - loss: 0.0410 - val_accuracy: 0.9897 -
val_loss: 0.0415 - learning_rate: 5.0000e-04
Epoch 27/30
844/844 - 4s - 5ms/step - accuracy: 0.9884 - loss: 0.0399 - val_accuracy: 0.9893 -
val_loss: 0.0417 - learning_rate: 5.0000e-04
Epoch 28/30
844/844 - 4s - 5ms/step - accuracy: 0.9880 - loss: 0.0400 - val_accuracy: 0.9892 -
val_loss: 0.0419 - learning_rate: 5.0000e-04
Epoch 29/30
844/844 - 4s - 5ms/step - accuracy: 0.9888 - loss: 0.0389 - val_accuracy: 0.9895 -
val_loss: 0.0416 - learning_rate: 2.5000e-04
Epoch 30/30
844/844 - 4s - 5ms/step - accuracy: 0.9891 - loss: 0.0383 - val_accuracy: 0.9897 -
val_loss: 0.0412 - learning_rate: 2.5000e-04
```

```
Training search_v2_wide_tanh {'active_size': 28, 'filters': [8, 24], 'activation':
'tanh', 'pooling': 'average', 'dropout': 0.1, 'learning_rate': 0.001, 'batch_size':
128, 'max_epochs': 30, 'translation': 0.0, 'reduce_lr': True, 'trial_id':
'wide_tanh'}
Epoch 1/30
422/422 - 5s - 11ms/step - accuracy: 0.8620 - loss: 0.4818 - val_accuracy: 0.9463 -
val_loss: 0.1982 - learning_rate: 0.0010
Epoch 2/30
422/422 - 3s - 7ms/step - accuracy: 0.9382 - loss: 0.2115 - val_accuracy: 0.9642 -
val_loss: 0.1330 - learning_rate: 0.0010
Epoch 3/30
422/422 - 3s - 7ms/step - accuracy: 0.9572 - loss: 0.1502 - val_accuracy: 0.9728 -
val_loss: 0.1027 - learning_rate: 0.0010
Epoch 4/30
422/422 - 3s - 8ms/step - accuracy: 0.9663 - loss: 0.1172 - val_accuracy: 0.9768 -
val_loss: 0.0856 - learning_rate: 0.0010
Epoch 5/30
422/422 - 3s - 8ms/step - accuracy: 0.9722 - loss: 0.0967 - val_accuracy: 0.9792 -
val_loss: 0.0747 - learning_rate: 0.0010
Epoch 6/30
422/422 - 3s - 8ms/step - accuracy: 0.9762 - loss: 0.0833 - val_accuracy: 0.9808 -
val_loss: 0.0663 - learning_rate: 0.0010
Epoch 7/30
422/422 - 3s - 8ms/step - accuracy: 0.9794 - loss: 0.0734 - val_accuracy: 0.9818 -
val_loss: 0.0613 - learning_rate: 0.0010
Epoch 8/30
422/422 - 3s - 8ms/step - accuracy: 0.9814 - loss: 0.0657 - val_accuracy: 0.9833 -
val_loss: 0.0566 - learning_rate: 0.0010
Epoch 9/30
422/422 - 3s - 8ms/step - accuracy: 0.9832 - loss: 0.0596 - val_accuracy: 0.9837 -
val_loss: 0.0534 - learning_rate: 0.0010
Epoch 10/30
422/422 - 3s - 7ms/step - accuracy: 0.9848 - loss: 0.0550 - val_accuracy: 0.9852 -
val_loss: 0.0506 - learning_rate: 0.0010
Epoch 11/30
422/422 - 3s - 8ms/step - accuracy: 0.9853 - loss: 0.0512 - val_accuracy: 0.9867 -
val_loss: 0.0481 - learning_rate: 0.0010
Epoch 12/30
422/422 - 3s - 8ms/step - accuracy: 0.9864 - loss: 0.0477 - val_accuracy: 0.9868 -
val_loss: 0.0462 - learning_rate: 0.0010
Epoch 13/30
422/422 - 3s - 8ms/step - accuracy: 0.9871 - loss: 0.0448 - val_accuracy: 0.9880 -
val_loss: 0.0446 - learning_rate: 0.0010
Epoch 14/30
422/422 - 3s - 7ms/step - accuracy: 0.9876 - loss: 0.0426 - val_accuracy: 0.9875 -
val_loss: 0.0436 - learning_rate: 0.0010
Epoch 15/30
422/422 - 3s - 8ms/step - accuracy: 0.9885 - loss: 0.0399 - val_accuracy: 0.9882 -
val_loss: 0.0430 - learning_rate: 0.0010
Epoch 16/30
422/422 - 3s - 7ms/step - accuracy: 0.9890 - loss: 0.0384 - val_accuracy: 0.9887 -
val_loss: 0.0415 - learning_rate: 0.0010
Epoch 17/30
422/422 - 3s - 8ms/step - accuracy: 0.9891 - loss: 0.0365 - val_accuracy: 0.9885 -
val_loss: 0.0403 - learning_rate: 0.0010
Epoch 18/30
```

```
422/422 - 3s - 8ms/step - accuracy: 0.9898 - loss: 0.0352 - val_accuracy: 0.9888 -
val_loss: 0.0394 - learning_rate: 0.0010
Epoch 19/30
422/422 - 3s - 7ms/step - accuracy: 0.9902 - loss: 0.0333 - val_accuracy: 0.9885 -
val_loss: 0.0391 - learning_rate: 0.0010
Epoch 20/30
422/422 - 3s - 7ms/step - accuracy: 0.9910 - loss: 0.0320 - val_accuracy: 0.9885 -
val_loss: 0.0381 - learning_rate: 0.0010
Epoch 21/30
422/422 - 3s - 8ms/step - accuracy: 0.9912 - loss: 0.0312 - val_accuracy: 0.9885 -
val_loss: 0.0384 - learning_rate: 0.0010
Epoch 22/30
422/422 - 3s - 8ms/step - accuracy: 0.9915 - loss: 0.0298 - val_accuracy: 0.9887 -
val_loss: 0.0382 - learning_rate: 0.0010
Epoch 23/30
422/422 - 3s - 8ms/step - accuracy: 0.9928 - loss: 0.0268 - val_accuracy: 0.9892 -
val_loss: 0.0362 - learning_rate: 5.0000e-04
Epoch 24/30
422/422 - 3s - 7ms/step - accuracy: 0.9930 - loss: 0.0259 - val_accuracy: 0.9895 -
val_loss: 0.0357 - learning_rate: 5.0000e-04
Epoch 25/30
422/422 - 3s - 7ms/step - accuracy: 0.9929 - loss: 0.0256 - val_accuracy: 0.9903 -
val_loss: 0.0358 - learning_rate: 5.0000e-04
Epoch 26/30
422/422 - 3s - 7ms/step - accuracy: 0.9932 - loss: 0.0251 - val_accuracy: 0.9902 -
val_loss: 0.0354 - learning_rate: 5.0000e-04
Epoch 27/30
422/422 - 3s - 8ms/step - accuracy: 0.9936 - loss: 0.0246 - val_accuracy: 0.9902 -
val_loss: 0.0355 - learning_rate: 5.0000e-04
Epoch 28/30
422/422 - 3s - 8ms/step - accuracy: 0.9935 - loss: 0.0243 - val_accuracy: 0.9898 -
val_loss: 0.0355 - learning_rate: 5.0000e-04
Epoch 29/30
422/422 - 3s - 8ms/step - accuracy: 0.9939 - loss: 0.0228 - val_accuracy: 0.9897 -
val_loss: 0.0356 - learning_rate: 2.5000e-04
Epoch 30/30
422/422 - 3s - 8ms/step - accuracy: 0.9941 - loss: 0.0225 - val_accuracy: 0.9902 -
val_loss: 0.0360 - learning_rate: 2.5000e-04
```

```
Training search_v2_wide_relu {'active_size': 28, 'filters': [8, 24], 'activation':
'relu', 'pooling': 'average', 'dropout': 0.1, 'learning_rate': 0.001, 'batch_size':
128, 'max_epochs': 30, 'translation': 0.0, 'reduce_lr': True, 'trial_id':
'wide_relu'}
Epoch 1/30
422/422 - 4s - 10ms/step - accuracy: 0.8650 - loss: 0.4601 - val_accuracy: 0.9620 -
val_loss: 0.1400 - learning_rate: 0.0010
Epoch 2/30
422/422 - 3s - 7ms/step - accuracy: 0.9514 - loss: 0.1632 - val_accuracy: 0.9760 -
val_loss: 0.0876 - learning_rate: 0.0010
Epoch 3/30
422/422 - 3s - 7ms/step - accuracy: 0.9670 - loss: 0.1128 - val_accuracy: 0.9808 -
val_loss: 0.0674 - learning_rate: 0.0010
Epoch 4/30
422/422 - 3s - 7ms/step - accuracy: 0.9735 - loss: 0.0893 - val_accuracy: 0.9840 -
val_loss: 0.0585 - learning_rate: 0.0010
Epoch 5/30
422/422 - 3s - 7ms/step - accuracy: 0.9781 - loss: 0.0745 - val_accuracy: 0.9870 -
val_loss: 0.0503 - learning_rate: 0.0010
Epoch 6/30
422/422 - 3s - 7ms/step - accuracy: 0.9800 - loss: 0.0661 - val_accuracy: 0.9870 -
val_loss: 0.0482 - learning_rate: 0.0010
Epoch 7/30
422/422 - 3s - 7ms/step - accuracy: 0.9824 - loss: 0.0585 - val_accuracy: 0.9880 -
val_loss: 0.0457 - learning_rate: 0.0010
Epoch 8/30
422/422 - 3s - 7ms/step - accuracy: 0.9831 - loss: 0.0551 - val_accuracy: 0.9887 -
val_loss: 0.0446 - learning_rate: 0.0010
Epoch 9/30
422/422 - 3s - 7ms/step - accuracy: 0.9853 - loss: 0.0494 - val_accuracy: 0.9882 -
val_loss: 0.0447 - learning_rate: 0.0010
Epoch 10/30
422/422 - 3s - 7ms/step - accuracy: 0.9854 - loss: 0.0466 - val_accuracy: 0.9885 -
val_loss: 0.0426 - learning_rate: 0.0010
Epoch 11/30
422/422 - 3s - 7ms/step - accuracy: 0.9862 - loss: 0.0437 - val_accuracy: 0.9887 -
val_loss: 0.0422 - learning_rate: 0.0010
Epoch 12/30
422/422 - 3s - 7ms/step - accuracy: 0.9871 - loss: 0.0406 - val_accuracy: 0.9893 -
val_loss: 0.0416 - learning_rate: 0.0010
Epoch 13/30
422/422 - 3s - 7ms/step - accuracy: 0.9876 - loss: 0.0390 - val_accuracy: 0.9888 -
val_loss: 0.0400 - learning_rate: 0.0010
Epoch 14/30
422/422 - 3s - 7ms/step - accuracy: 0.9888 - loss: 0.0358 - val_accuracy: 0.9882 -
val_loss: 0.0430 - learning_rate: 0.0010
Epoch 15/30
422/422 - 3s - 7ms/step - accuracy: 0.9890 - loss: 0.0343 - val_accuracy: 0.9888 -
val_loss: 0.0396 - learning_rate: 0.0010
Epoch 16/30
422/422 - 3s - 7ms/step - accuracy: 0.9898 - loss: 0.0328 - val_accuracy: 0.9905 -
val_loss: 0.0359 - learning_rate: 0.0010
Epoch 17/30
422/422 - 3s - 7ms/step - accuracy: 0.9900 - loss: 0.0311 - val_accuracy: 0.9887 -
val_loss: 0.0384 - learning_rate: 0.0010
Epoch 18/30
```

```
422/422 - 3s - 7ms/step - accuracy: 0.9906 - loss: 0.0303 - val_accuracy: 0.9893 -
val_loss: 0.0394 - learning_rate: 0.0010
Epoch 19/30
422/422 - 3s - 7ms/step - accuracy: 0.9921 - loss: 0.0255 - val_accuracy: 0.9908 -
val_loss: 0.0376 - learning_rate: 5.0000e-04
Epoch 20/30
422/422 - 3s - 7ms/step - accuracy: 0.9921 - loss: 0.0252 - val_accuracy: 0.9902 -
val_loss: 0.0371 - learning_rate: 5.0000e-04
Epoch 21/30
422/422 - 3s - 7ms/step - accuracy: 0.9933 - loss: 0.0218 - val_accuracy: 0.9905 -
val_loss: 0.0361 - learning_rate: 2.5000e-04
Epoch 22/30
422/422 - 3s - 7ms/step - accuracy: 0.9932 - loss: 0.0221 - val_accuracy: 0.9900 -
val_loss: 0.0351 - learning_rate: 2.5000e-04
Epoch 23/30
422/422 - 3s - 7ms/step - accuracy: 0.9930 - loss: 0.0211 - val_accuracy: 0.9900 -
val_loss: 0.0374 - learning_rate: 2.5000e-04
Epoch 24/30
422/422 - 3s - 7ms/step - accuracy: 0.9935 - loss: 0.0208 - val_accuracy: 0.9908 -
val_loss: 0.0357 - learning_rate: 2.5000e-04
Epoch 25/30
422/422 - 3s - 7ms/step - accuracy: 0.9938 - loss: 0.0200 - val_accuracy: 0.9903 -
val_loss: 0.0368 - learning_rate: 1.2500e-04
Epoch 26/30
422/422 - 3s - 7ms/step - accuracy: 0.9936 - loss: 0.0195 - val_accuracy: 0.9902 -
val_loss: 0.0368 - learning_rate: 1.2500e-04
Epoch 27/30
422/422 - 3s - 7ms/step - accuracy: 0.9941 - loss: 0.0191 - val_accuracy: 0.9900 -
val_loss: 0.0364 - learning_rate: 6.2500e-05
Epoch 28/30
422/422 - 3s - 7ms/step - accuracy: 0.9943 - loss: 0.0187 - val_accuracy: 0.9902 -
val_loss: 0.0362 - learning_rate: 6.2500e-05
```

```
Training search_v2_wide_shift {'active_size': 28, 'filters': [8, 24], 'activation':
'tanh', 'pooling': 'average', 'dropout': 0.1, 'learning_rate': 0.001, 'batch_size':
128, 'max_epochs': 30, 'translation': 0.07, 'reduce_lr': True, 'trial_id':
'wide_shift'}
Epoch 1/30
422/422 - 6s - 14ms/step - accuracy: 0.7566 - loss: 0.7795 - val_accuracy: 0.9440 -
val_loss: 0.2094 - learning_rate: 0.0010
Epoch 2/30
422/422 - 4s - 10ms/step - accuracy: 0.9029 - loss: 0.3420 - val_accuracy: 0.9693 -
val_loss: 0.1273 - learning_rate: 0.0010
Epoch 3/30
422/422 - 4s - 10ms/step - accuracy: 0.9376 - loss: 0.2264 - val_accuracy: 0.9747 -
val_loss: 0.0974 - learning_rate: 0.0010
Epoch 4/30
422/422 - 4s - 10ms/step - accuracy: 0.9504 - loss: 0.1773 - val_accuracy: 0.9793 -
val_loss: 0.0813 - learning_rate: 0.0010
Epoch 5/30
422/422 - 4s - 10ms/step - accuracy: 0.9584 - loss: 0.1477 - val_accuracy: 0.9817 -
val_loss: 0.0711 - learning_rate: 0.0010
Epoch 6/30
422/422 - 4s - 10ms/step - accuracy: 0.9619 - loss: 0.1325 - val_accuracy: 0.9818 -
val_loss: 0.0636 - learning_rate: 0.0010
Epoch 7/30
422/422 - 4s - 10ms/step - accuracy: 0.9660 - loss: 0.1170 - val_accuracy: 0.9835 -
val_loss: 0.0590 - learning_rate: 0.0010
Epoch 8/30
422/422 - 4s - 10ms/step - accuracy: 0.9689 - loss: 0.1062 - val_accuracy: 0.9852 -
val_loss: 0.0529 - learning_rate: 0.0010
Epoch 9/30
422/422 - 4s - 10ms/step - accuracy: 0.9716 - loss: 0.0995 - val_accuracy: 0.9852 -
val_loss: 0.0503 - learning_rate: 0.0010
Epoch 10/30
422/422 - 4s - 10ms/step - accuracy: 0.9729 - loss: 0.0936 - val_accuracy: 0.9857 -
val_loss: 0.0484 - learning_rate: 0.0010
Epoch 11/30
422/422 - 4s - 10ms/step - accuracy: 0.9740 - loss: 0.0878 - val_accuracy: 0.9863 -
val_loss: 0.0473 - learning_rate: 0.0010
Epoch 12/30
422/422 - 4s - 10ms/step - accuracy: 0.9759 - loss: 0.0824 - val_accuracy: 0.9880 -
val_loss: 0.0444 - learning_rate: 0.0010
Epoch 13/30
422/422 - 4s - 10ms/step - accuracy: 0.9763 - loss: 0.0798 - val_accuracy: 0.9882 -
val_loss: 0.0418 - learning_rate: 0.0010
Epoch 14/30
422/422 - 4s - 10ms/step - accuracy: 0.9769 - loss: 0.0770 - val_accuracy: 0.9875 -
val_loss: 0.0419 - learning_rate: 0.0010
Epoch 15/30
422/422 - 4s - 10ms/step - accuracy: 0.9775 - loss: 0.0748 - val_accuracy: 0.9877 -
val_loss: 0.0414 - learning_rate: 0.0010
Epoch 16/30
422/422 - 4s - 10ms/step - accuracy: 0.9786 - loss: 0.0717 - val_accuracy: 0.9892 -
val_loss: 0.0388 - learning_rate: 0.0010
Epoch 17/30
422/422 - 4s - 10ms/step - accuracy: 0.9788 - loss: 0.0691 - val_accuracy: 0.9893 -
val_loss: 0.0384 - learning_rate: 0.0010
Epoch 18/30
```

```
422/422 - 4s - 10ms/step - accuracy: 0.9798 - loss: 0.0666 - val_accuracy: 0.9900 -
val_loss: 0.0375 - learning_rate: 0.0010
Epoch 19/30
422/422 - 4s - 10ms/step - accuracy: 0.9800 - loss: 0.0644 - val_accuracy: 0.9900 -
val_loss: 0.0364 - learning_rate: 0.0010
Epoch 20/30
422/422 - 4s - 10ms/step - accuracy: 0.9802 - loss: 0.0641 - val_accuracy: 0.9898 -
val_loss: 0.0374 - learning_rate: 0.0010
Epoch 21/30
422/422 - 4s - 10ms/step - accuracy: 0.9808 - loss: 0.0608 - val_accuracy: 0.9912 -
val_loss: 0.0349 - learning_rate: 0.0010
Epoch 22/30
422/422 - 4s - 10ms/step - accuracy: 0.9814 - loss: 0.0608 - val_accuracy: 0.9910 -
val_loss: 0.0351 - learning_rate: 0.0010
Epoch 23/30
422/422 - 4s - 10ms/step - accuracy: 0.9815 - loss: 0.0597 - val_accuracy: 0.9912 -
val_loss: 0.0337 - learning_rate: 0.0010
Epoch 24/30
422/422 - 4s - 10ms/step - accuracy: 0.9815 - loss: 0.0586 - val_accuracy: 0.9910 -
val_loss: 0.0330 - learning_rate: 0.0010
Epoch 25/30
422/422 - 4s - 10ms/step - accuracy: 0.9830 - loss: 0.0553 - val_accuracy: 0.9912 -
val_loss: 0.0337 - learning_rate: 0.0010
Epoch 26/30
422/422 - 4s - 10ms/step - accuracy: 0.9834 - loss: 0.0562 - val_accuracy: 0.9925 -
val_loss: 0.0313 - learning_rate: 0.0010
Epoch 27/30
422/422 - 4s - 10ms/step - accuracy: 0.9829 - loss: 0.0545 - val_accuracy: 0.9905 -
val_loss: 0.0355 - learning_rate: 0.0010
Epoch 28/30
422/422 - 4s - 10ms/step - accuracy: 0.9838 - loss: 0.0532 - val_accuracy: 0.9908 -
val_loss: 0.0340 - learning_rate: 0.0010
Epoch 29/30
422/422 - 4s - 10ms/step - accuracy: 0.9851 - loss: 0.0492 - val_accuracy: 0.9918 -
val_loss: 0.0312 - learning_rate: 5.0000e-04
Epoch 30/30
422/422 - 4s - 10ms/step - accuracy: 0.9851 - loss: 0.0492 - val_accuracy: 0.9922 -
val_loss: 0.0305 - learning_rate: 5.0000e-04
```

```
Training search_v2_wide_relu_max {'active_size': 28, 'filters': [8, 24],
'activation': 'relu', 'pooling': 'max', 'dropout': 0.2, 'learning_rate': 0.001,
'batch_size': 128, 'max_epochs': 30, 'translation': 0.07, 'reduce_lr': True,
'trial_id': 'wide_relu_max'}
Epoch 1/30
422/422 - 6s - 14ms/step - accuracy: 0.8223 - loss: 0.5823 - val_accuracy: 0.9697 -
val_loss: 0.1009 - learning_rate: 0.0010
Epoch 2/30
422/422 - 4s - 10ms/step - accuracy: 0.9422 - loss: 0.1978 - val_accuracy: 0.9812 -
val_loss: 0.0687 - learning_rate: 0.0010
Epoch 3/30
422/422 - 4s - 10ms/step - accuracy: 0.9561 - loss: 0.1493 - val_accuracy: 0.9818 -
val_loss: 0.0630 - learning_rate: 0.0010
Epoch 4/30
422/422 - 4s - 10ms/step - accuracy: 0.9634 - loss: 0.1226 - val_accuracy: 0.9835 -
val_loss: 0.0535 - learning_rate: 0.0010
Epoch 5/30
422/422 - 4s - 10ms/step - accuracy: 0.9677 - loss: 0.1084 - val_accuracy: 0.9855 -
val_loss: 0.0461 - learning_rate: 0.0010
Epoch 6/30
422/422 - 4s - 10ms/step - accuracy: 0.9697 - loss: 0.1015 - val_accuracy: 0.9872 -
val_loss: 0.0439 - learning_rate: 0.0010
Epoch 7/30
422/422 - 4s - 10ms/step - accuracy: 0.9720 - loss: 0.0949 - val_accuracy: 0.9868 -
val_loss: 0.0420 - learning_rate: 0.0010
Epoch 8/30
422/422 - 4s - 10ms/step - accuracy: 0.9732 - loss: 0.0872 - val_accuracy: 0.9875 -
val_loss: 0.0406 - learning_rate: 0.0010
Epoch 9/30
422/422 - 4s - 10ms/step - accuracy: 0.9759 - loss: 0.0814 - val_accuracy: 0.9883 -
val_loss: 0.0391 - learning_rate: 0.0010
Epoch 10/30
422/422 - 4s - 10ms/step - accuracy: 0.9761 - loss: 0.0808 - val_accuracy: 0.9883 -
val_loss: 0.0407 - learning_rate: 0.0010
Epoch 11/30
422/422 - 4s - 10ms/step - accuracy: 0.9775 - loss: 0.0739 - val_accuracy: 0.9883 -
val_loss: 0.0391 - learning_rate: 0.0010
Epoch 12/30
422/422 - 4s - 10ms/step - accuracy: 0.9796 - loss: 0.0695 - val_accuracy: 0.9890 -
val_loss: 0.0353 - learning_rate: 5.0000e-04
Epoch 13/30
422/422 - 4s - 10ms/step - accuracy: 0.9803 - loss: 0.0671 - val_accuracy: 0.9905 -
val_loss: 0.0370 - learning_rate: 5.0000e-04
Epoch 14/30
422/422 - 4s - 10ms/step - accuracy: 0.9798 - loss: 0.0663 - val_accuracy: 0.9900 -
val_loss: 0.0357 - learning_rate: 5.0000e-04
Epoch 15/30
422/422 - 4s - 11ms/step - accuracy: 0.9813 - loss: 0.0622 - val_accuracy: 0.9907 -
val_loss: 0.0333 - learning_rate: 2.5000e-04
Epoch 16/30
422/422 - 4s - 10ms/step - accuracy: 0.9819 - loss: 0.0607 - val_accuracy: 0.9913 -
val_loss: 0.0321 - learning_rate: 2.5000e-04
Epoch 17/30
422/422 - 4s - 10ms/step - accuracy: 0.9820 - loss: 0.0595 - val_accuracy: 0.9910 -
val_loss: 0.0331 - learning_rate: 2.5000e-04
Epoch 18/30
```

```
422/422 - 4s - 10ms/step - accuracy: 0.9818 - loss: 0.0596 - val_accuracy: 0.9907 -
val_loss: 0.0329 - learning_rate: 2.5000e-04
Epoch 19/30
422/422 - 4s - 10ms/step - accuracy: 0.9827 - loss: 0.0577 - val_accuracy: 0.9905 -
val_loss: 0.0330 - learning_rate: 1.2500e-04
Epoch 20/30
422/422 - 4s - 10ms/step - accuracy: 0.9826 - loss: 0.0576 - val_accuracy: 0.9905 -
val_loss: 0.0327 - learning_rate: 1.2500e-04
Epoch 21/30
422/422 - 4s - 10ms/step - accuracy: 0.9834 - loss: 0.0551 - val_accuracy: 0.9912 -
val_loss: 0.0323 - learning_rate: 6.2500e-05
Epoch 22/30
422/422 - 4s - 10ms/step - accuracy: 0.9836 - loss: 0.0553 - val_accuracy: 0.9908 -
val_loss: 0.0326 - learning_rate: 6.2500e-05

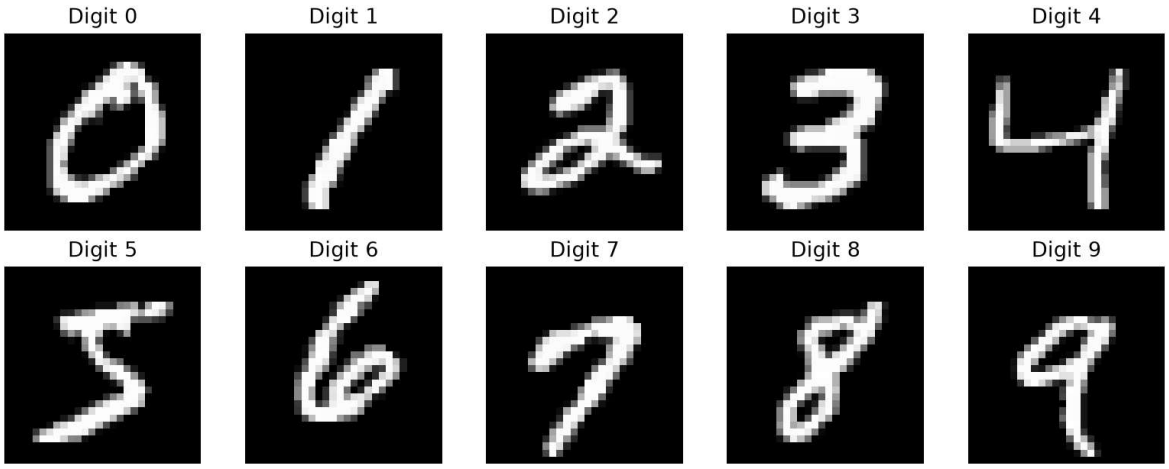
Selected on validation loss: search_v2_wide_shift {'active_size': 28, 'filters': [8,
24], 'activation': 'tanh', 'pooling': 'average', 'dropout': 0.1, 'learning_rate':
0.001, 'batch_size': 128, 'max_epochs': 30, 'translation': 0.07, 'reduce_lr': True,
'trial_id': 'wide_shift'}
Loaded selected trial: search_v2_wide_shift
Verified test accuracy: 99.09%
```

In [65]:
```python
# Detecting handwritten digits inside images
RECOGNITION_DIR.mkdir(parents=True, exist_ok=True)
records = []
models = {name: keras.models.load_model(MODEL_DIRS[name] / f'{name}.keras') for
name in MODEL_NAMES}
for path in TEST_IMAGES:
    digits, gray, boxes = extract_digits(path)
    if len(digits) == 0:
        print(f'No digits detected in {path.name}; skipping.')
        continue
    preview = cv2.cvtColor(gray, cv2.COLOR_GRAY2BGR)
    for x, y, w, h in boxes:
        cv2.rectangle(preview, (x, y), (x+w, y+h), (0, 255, 0), 2)
    cv2.imwrite(str(RECOGNITION_DIR / f'{path.stem}_detected.png'), preview)
    for name, model in models.items():
        # Using the same image preparation as training, disabling dropout and
random shifts
        probs = model(tf.convert_to_tensor(preprocess(digits, name),
dtype=tf.float32), training=False).numpy()
        prediction = ''.join(map(str, probs.argmax(axis=1)))
        records.append({'image': path.name, 'model': name, 'prediction':
prediction, 'confidence': float(probs.max(axis=1).mean())})
recognition_results = pd.DataFrame(records, columns=['image', 'model',
'prediction', 'confidence'])
recognition_results.to_csv(RECOGNITION_DIR / 'predictions.csv', index=False)
```

5- Results and Comparison: chollet, lecun, lecun_tune

In [66]:
```python
# MNIST samples
fig, axs = plt.subplots(2, 5, figsize=(10, 4))
for digit, ax in enumerate(axs.flat):
```

```
    idx = np.flatnonzero(ya == digit)[0]
    ax.imshow(xa[idx], cmap='gray')
    ax.set_title(f'Digit {digit}')
    ax.axis('off')
fig.tight_layout()
fig.savefig(RUN_DIR / 'mnist_examples.png', dpi=160)
plt.close(fig)
display(Image(filename=str(RUN_DIR / 'mnist_examples.png')))
```



In [67]:
```
# Loading results
results = pd.concat([pd.read_csv(comparison_path).query('model in
@BASE_MODEL_NAMES'), pd.DataFrame([tuned_result])], ignore_index=True)
#print('Results loaded from:', RUN_DIR)
#display(results.style.format({'test_loss': '{:.4f}', 'test_accuracy': '{:.2%}',
'training_seconds': '{:.1f}'}))
results.to_csv(RUN_DIR / 'comparison.csv', index=False)
summary = '# \n'
summary += '| Model | Test loss | Test accuracy | Training seconds | Parameters
|\n|---|---:|---:|---:|---:|\n'
for row in results.to_dict('records'):
    summary += f"| {row['model']} | {row['test_loss']:.5f} |
{row['test_accuracy']:.2%} | {row['training_seconds']:.1f} | {row['parameters']}
|\n"
display(Markdown(summary))
```

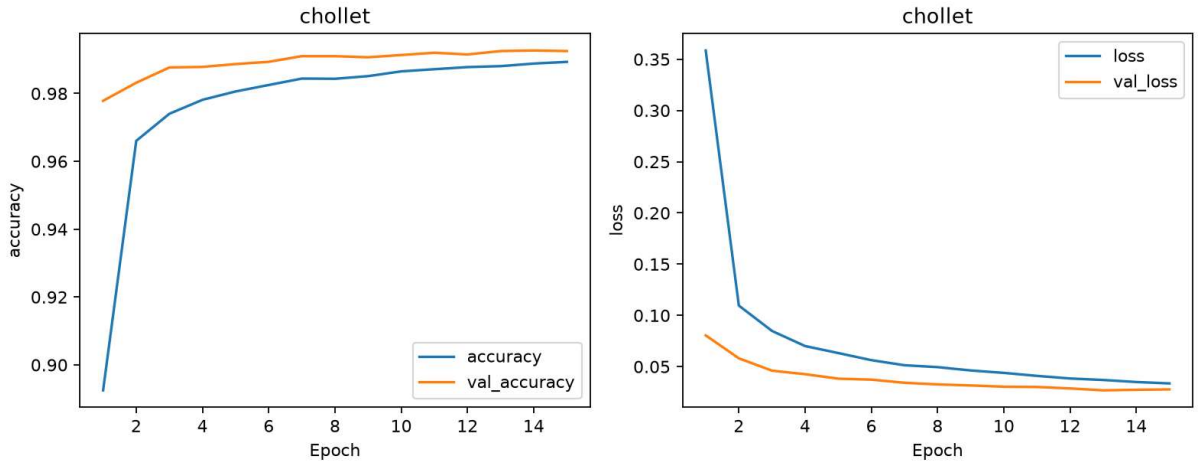
Model	Test loss	Test accuracy	Training seconds	Parameters
chollet	0.02596	99.17%	156.5	34826
lecun	0.10544	96.50%	52.8	3246
lecun_tune	0.02842	99.09%	621.1	8882

In [68]:
```
# Trials validation leaderboard
display(tuning_results.drop(columns=['model_path',
'history_path','config.trial_id'], errors='ignore'))
```

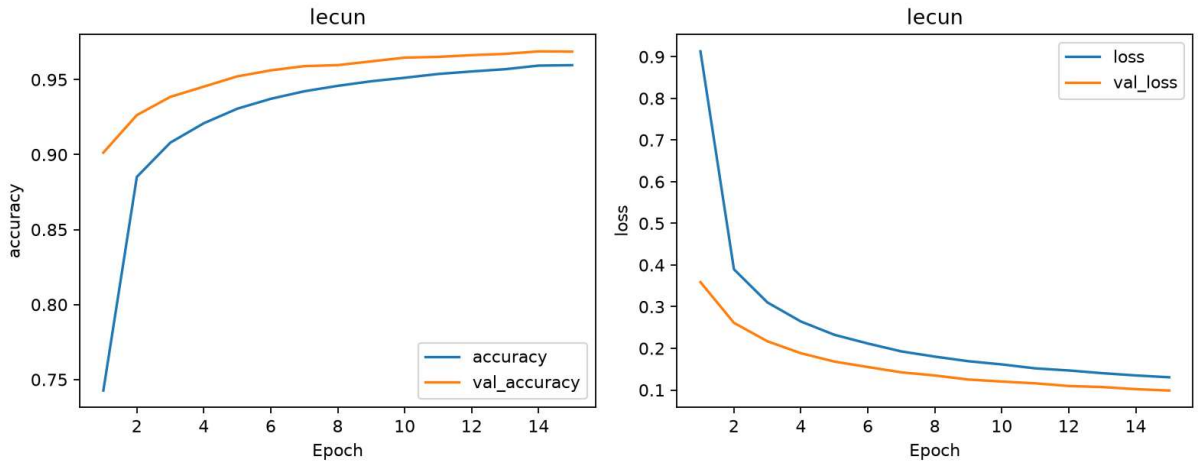
	trial_id	val_loss	val_accuracy	training_seconds	epochs_run	best_epoch
0	search_v2_wide_shift	0.030541	0.992167	130.400850	30	30
1	search_v2_wide_relu_max	0.032069	0.991333	94.988957	22	16
2	search_v2_wide_relu	0.035149	0.990000	85.887609	28	22
3	search_v2_wide_tanh	0.035352	0.990167	97.974737	30	26
4	search_v2_small_long	0.040434	0.988000	85.050137	30	30
5	search_v2_small_batch64	0.041199	0.989667	126.797477	30	30

In [69]:
```
# Learning curves comparison
for name in MODEL_NAMES:
    display(Markdown(f'**{name}**'))
    path = MODEL_DIRS[name] / f'{name}_learning.png'
    if path.exists():
        display(Image(filename=str(path)))
```

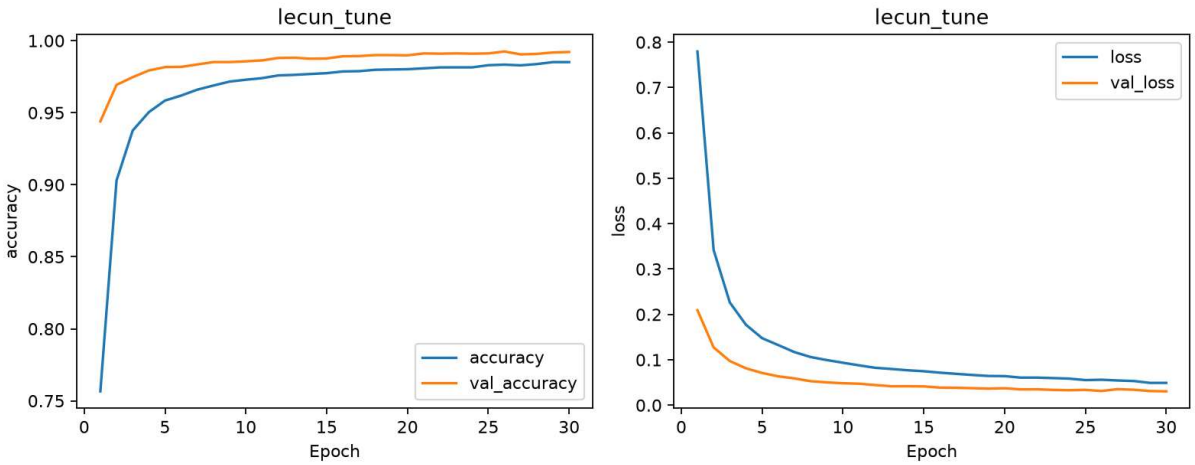
chollet



lecun

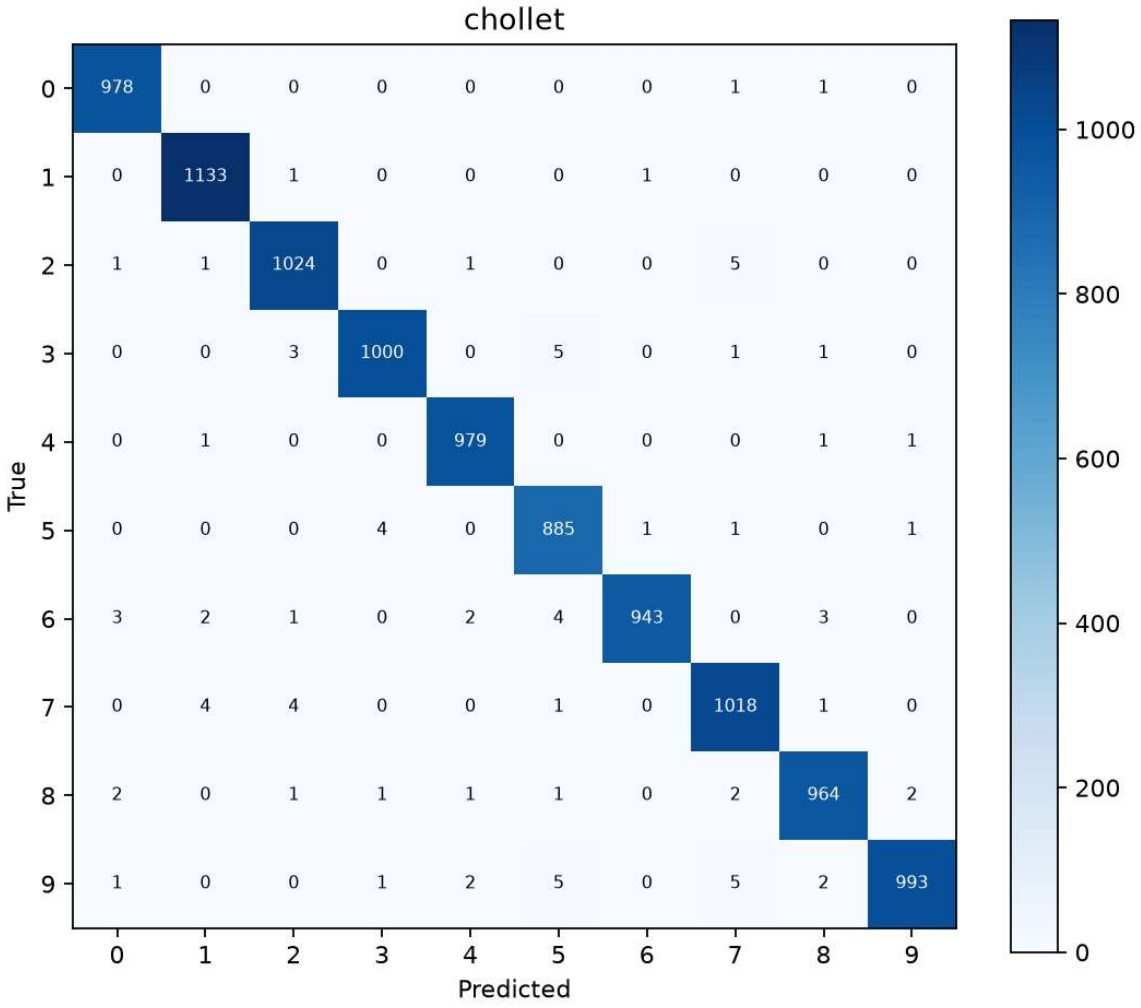


lecun_tune

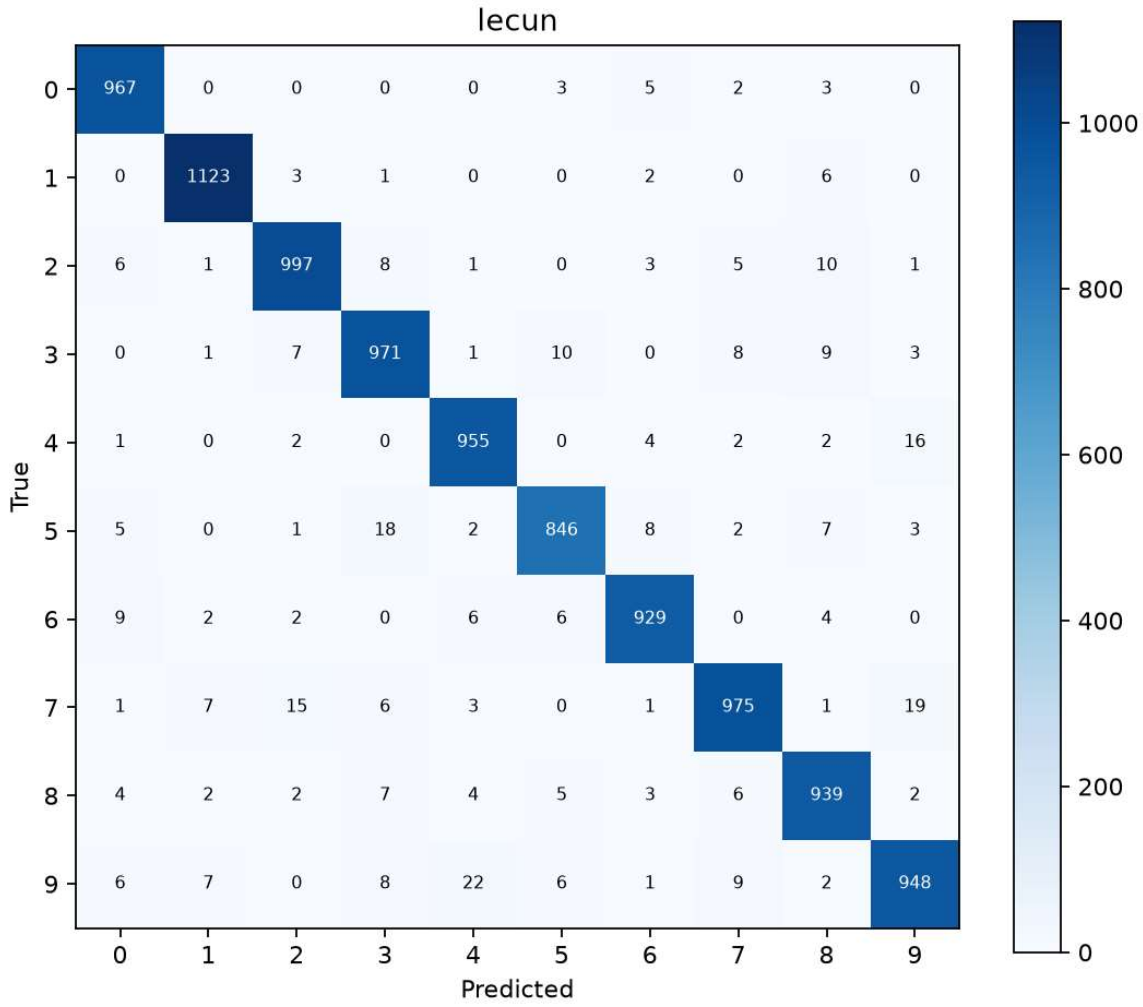


In [70]:
```
# Confusion matrices comparison
for name in MODEL_NAMES:
    display(Markdown(f'**{name}**'))
    path = MODEL_DIRS[name] / f'{name}_confusion.png'
    if path.exists():
        display(Image(filename=str(path)))
```

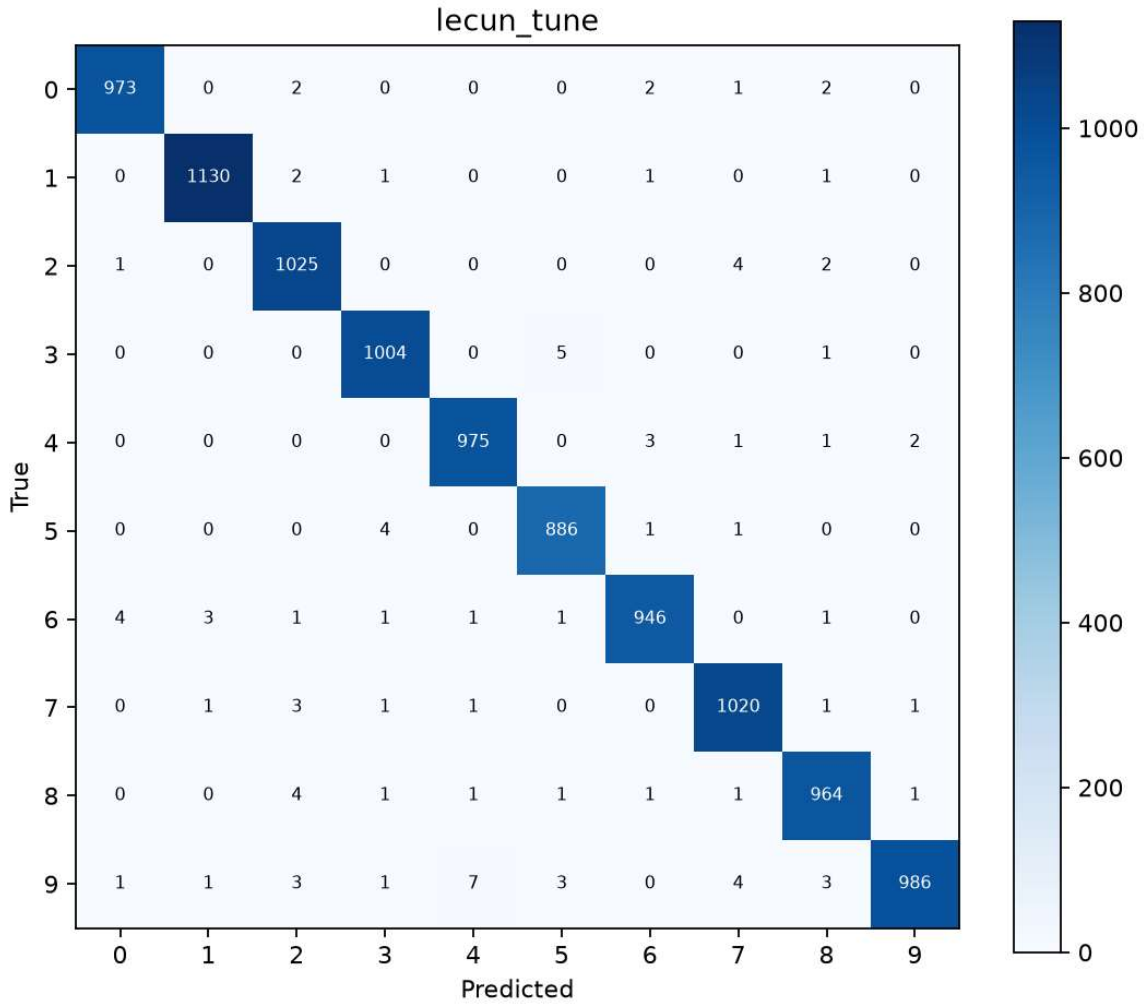
chollet



lecun

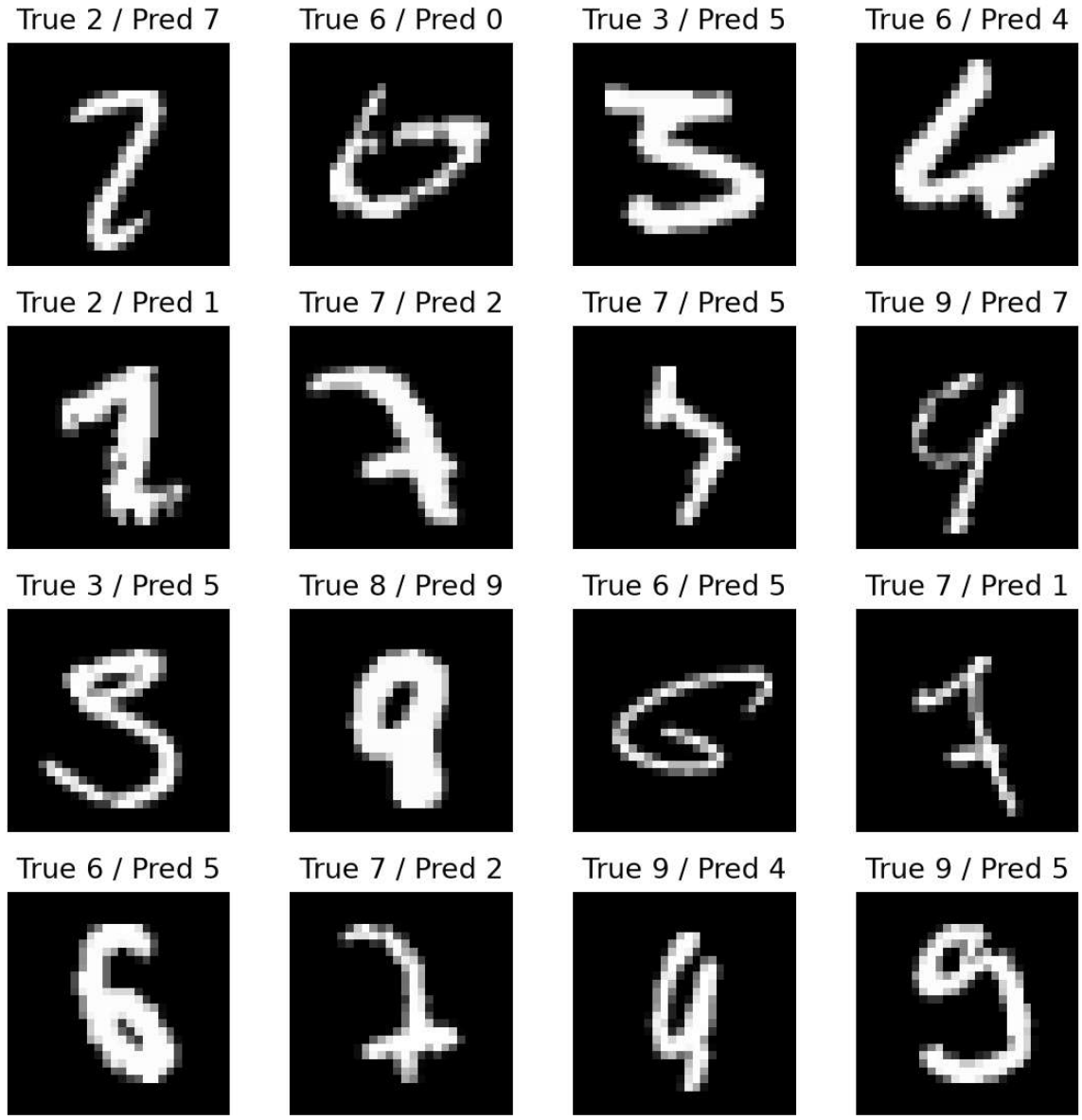


lecun_tune

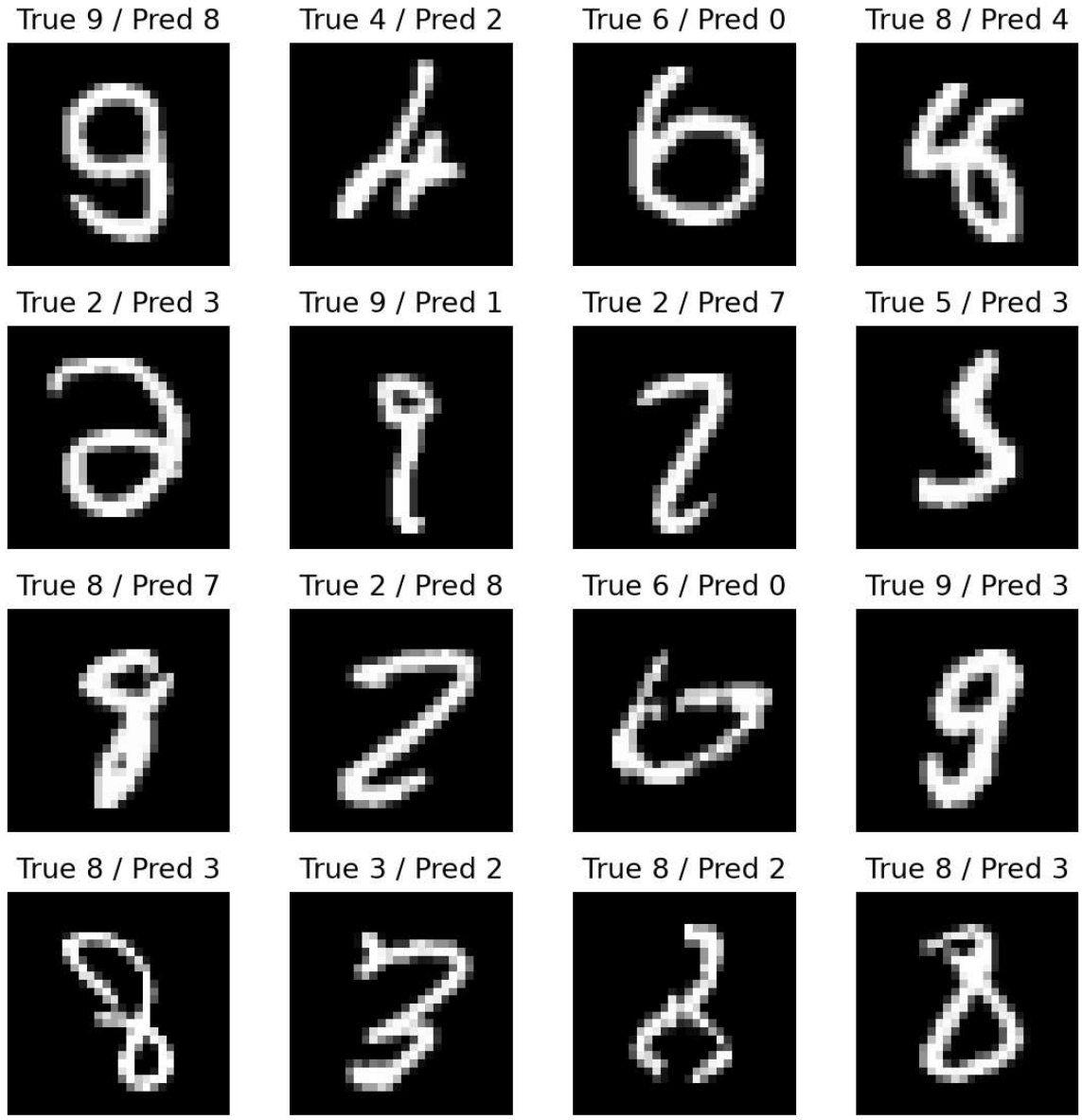


In [71]:
```
# Misclassifications comparison
for name in MODEL_NAMES:
    display(Markdown(f'**{name}**'))
    path = MODEL_DIRS[name] / f'{name}_errors.png'
    if path.exists():
        display(Image(filename=str(path)))
```

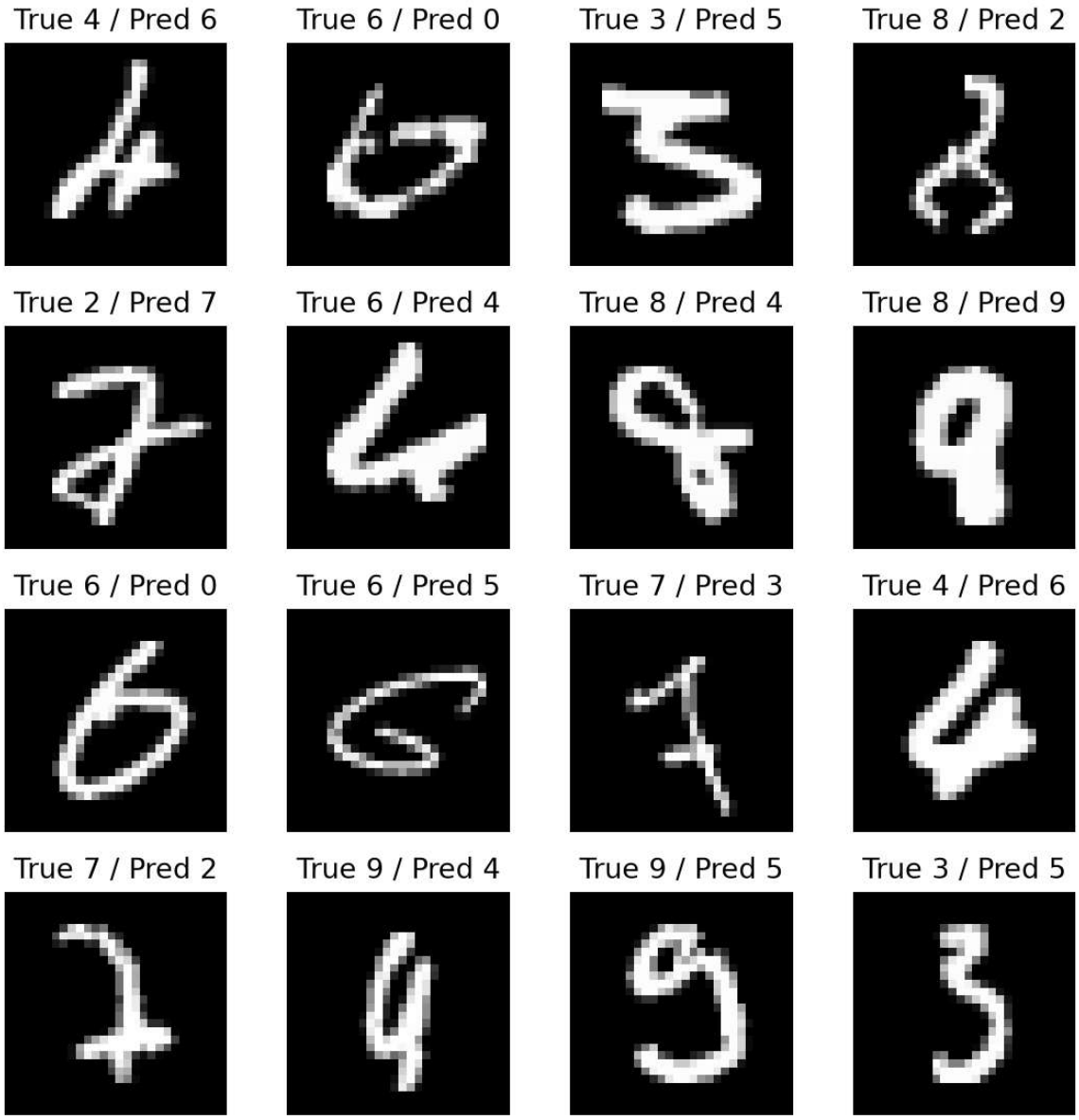
chollet


True 2 / Pred 7
True 6 / Pred 0
True 3 / Pred 5
True 6 / Pred 4
True 2 / Pred 1
True 7 / Pred 2
True 7 / Pred 5
True 9 / Pred 7
True 3 / Pred 5
True 8 / Pred 9
True 6 / Pred 5
True 7 / Pred 1
True 6 / Pred 5
True 7 / Pred 2
True 9 / Pred 4
True 9 / Pred 5

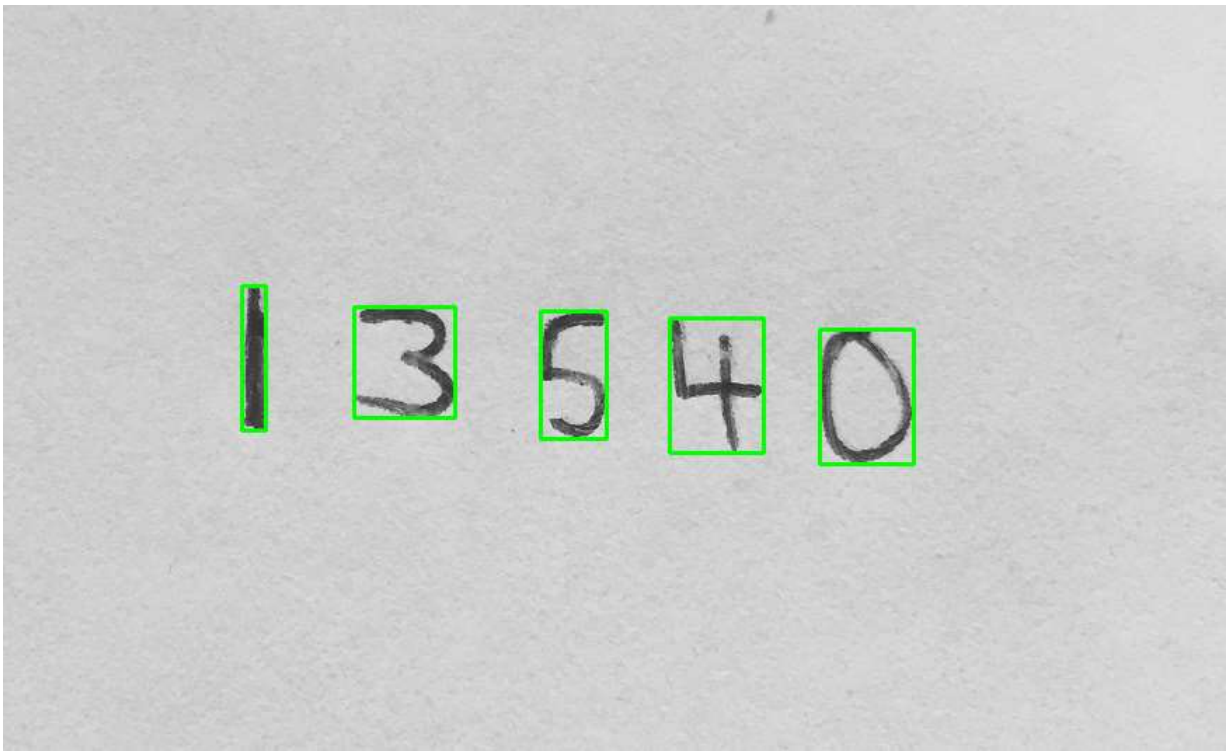
lecun


True 9 / Pred 8
True 4 / Pred 2
True 6 / Pred 0
True 8 / Pred 4
True 2 / Pred 3
True 9 / Pred 1
True 2 / Pred 7
True 5 / Pred 3
True 8 / Pred 7
True 2 / Pred 8
True 6 / Pred 0
True 9 / Pred 3
True 8 / Pred 3
True 3 / Pred 2
True 8 / Pred 2
True 8 / Pred 3

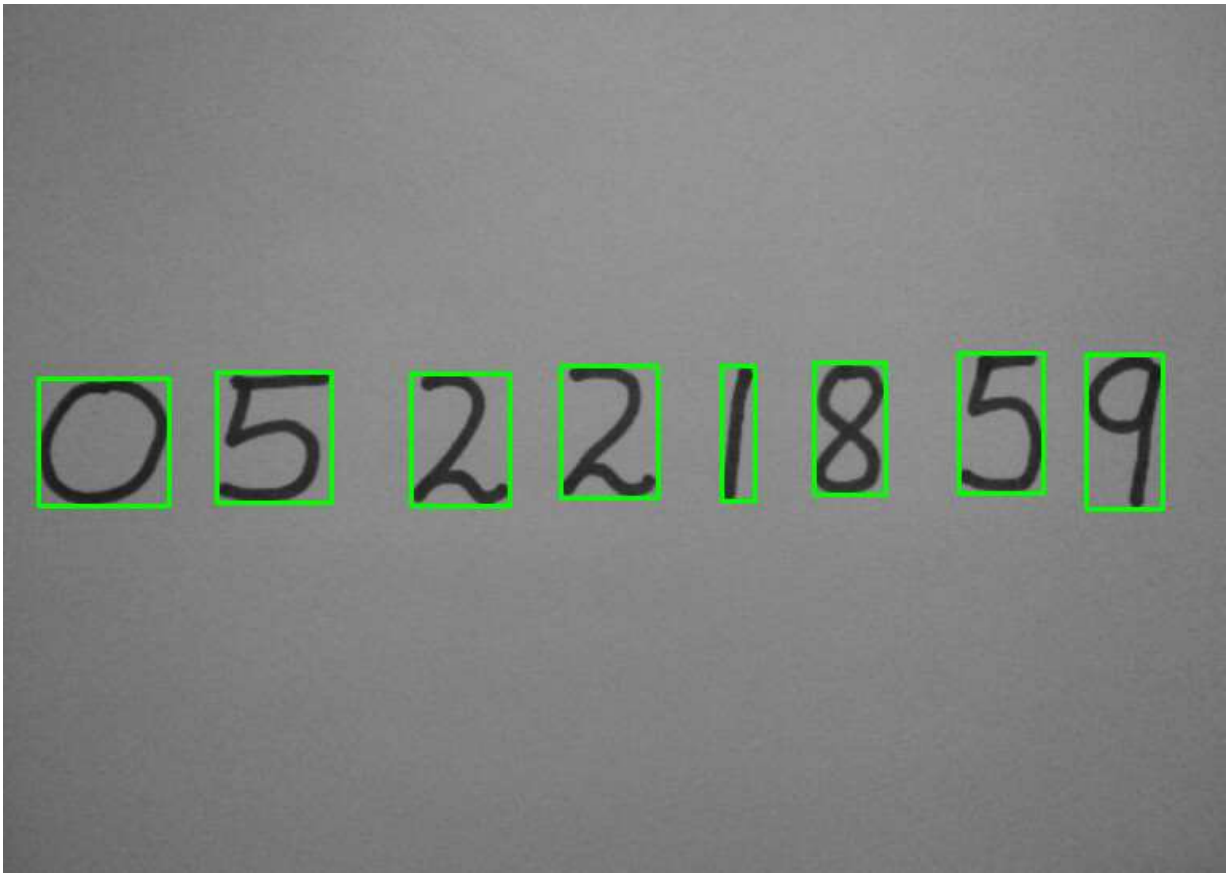
lecun_tune



In [72]:
```
# Handwritten rigit recognition results
for path in TEST_IMAGES:
    if not (RECOGNITION_DIR / f'{path.stem}_detected.png').exists():
        continue
    display(Image(filename=str(RECOGNITION_DIR / f'{path.stem}_detected.png')))
    results = recognition_results[recognition_results['image'] == path.name]
    display(results[['model', 'prediction',
'confidence']].style.hide(axis='index').format({'confidence': '{:.2%}'}))
```



model	prediction	confidence
chollet	13540	99.99%
lecun	13540	95.49%
lecun_tune	13540	99.89%



model	prediction	confidence
chollet	05221859	99.85%
lecun	05221859	96.80%
lecun_tune	05221859	99.90%